



 @azdocuments

PROGRAMMING IN C

SUBJECT CODE: 1BEIT105/205

MODULE-1

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

PROGRAMMING IN C

1BEIT105/205

MODULE-01

Introduction to Computing: Computer languages, Creating and Running Programs, System Development.

Overview of C: A Brief History of C, C Is a Middle-Level Language, C Is a Structured Language, C Is a Programmer's Language, Compilers Vs. Interpreters, The Form of a C Program, The Library and Linking, Separate Compilation, Compiling a C Program, C's Memory Map.

Expressions: The Basic Data Types, Modifying the Basic Types, Identifier Names, Variables, The Four C Scopes, Type Qualifiers, Storage Class Specifiers, Variable Initializations, Constants, Operators, Expressions.

Textbook 2: Chapter 1: 1.3, 1.4, 1.5; Textbook 1: Chapter 1, 2

COMPUTER SYSTEMS

Introduction

Computer Science is a field that studies computers, computational systems, and how problems can be solved using computer programs. It plays a major role in modern technology and is used in almost every field such as education, healthcare, business, communication, banking, transportation, and research.

A computer system helps humans perform complex tasks quickly and accurately. In the field of programming, it is important to understand how a computer system works because programs interact directly with the components of the system.

This chapter introduces the **basic concepts of computer systems**, including the hardware and software components. It also discusses the development of programming languages, the importance of the C programming language, and the tools used for writing computer programs.

Definition of a Computer

A **computer** is an electronic programmable machine that accepts data as input, processes the data according to a set of instructions, and produces meaningful results as output.

In simple terms:

Computer = Input + Processing + Output + Storage

A computer performs four basic operations:

1. **Input** – Accepting data from the user.
2. **Processing** – Performing calculations or logical operations.
3. **Output** – Producing results.
4. **Storage** – Storing data and programs for future use.

Computer System

A **computer system** is a combination of hardware and software components that work together to perform computational tasks.

The two main components of a computer system are:

1. Hardware
2. Software

Both components are equally important and depend on each other for proper functioning.

Components of a Computer System

1. Hardware

Hardware refers to the **physical components of a computer system that can be seen and touched**. These components are responsible for performing operations such as input, processing, storage, and output.

Major Hardware Components

1. Central Processing Unit (CPU)

The CPU is considered the **brain of the computer** because it controls all the operations performed by the computer.

The CPU consists of three main parts:

a) Arithmetic Logic Unit (ALU)

The ALU performs arithmetic operations such as addition, subtraction, multiplication, and division. It also performs logical operations such as comparison and decision making.

b) Control Unit (CU)

The Control Unit manages and coordinates the activities of all components of the computer. It directs the flow of data between the CPU, memory, and input/output devices.

c) Registers

Registers are small, high-speed storage locations inside the CPU used to temporarily hold data during processing.

2. Input Devices

Input devices are used to enter data and instructions into the computer system.

Examples include:

- Keyboard
- Mouse
- Scanner
- Microphone
- Webcam

These devices convert user actions into signals that the computer can process.

3. Output Devices

Output devices display or present the processed results to the user.

Examples include:

- Monitor
- Printer
- Speakers
- Projector

These devices convert processed data into human-readable form.

4. Memory and Storage Devices

Memory is used to store data and instructions required for processing.

Memory is divided into two types:

Primary Memory (Main Memory)

Primary memory stores data that is currently being processed.

Examples:

- RAM (Random Access Memory)
- ROM (Read Only Memory)

RAM is volatile memory, meaning data is lost when power is turned off.

Secondary Memory

Secondary memory stores data permanently for future use.

Examples:

- Hard Disk
- Solid State Drive (SSD)
- USB Flash Drive
- CD/DVD

Secondary memory has larger storage capacity compared to primary memory.

2. Software

Software is a collection of programs and instructions that tell the hardware how to perform specific tasks.

Without software, the hardware cannot perform any meaningful operation.

Examples of software include:

- Web browsers
- MS Word
- MS Excel
- Windows operating system

Software acts as an interface between the **user and the hardware**.

Types of Software

Software can be broadly classified into two categories.

1. System Software

System software controls the operation of the computer hardware and provides a platform for running application programs.

Examples include:

- Operating systems (Windows, Linux, macOS)
- Device drivers
- Utility programs

Functions of system software:

- Manages hardware resources
- Controls memory usage
- Handles input and output devices
- Provides security and system maintenance

2. Application Software

Application software is designed to perform specific tasks for users.

Examples include:

- Microsoft Word – document creation
- Microsoft Excel – spreadsheet calculations
- Web browsers – internet browsing
- Media players – playing audio and video

Application software makes computers useful for practical everyday tasks.

Relationship Between Hardware and Software

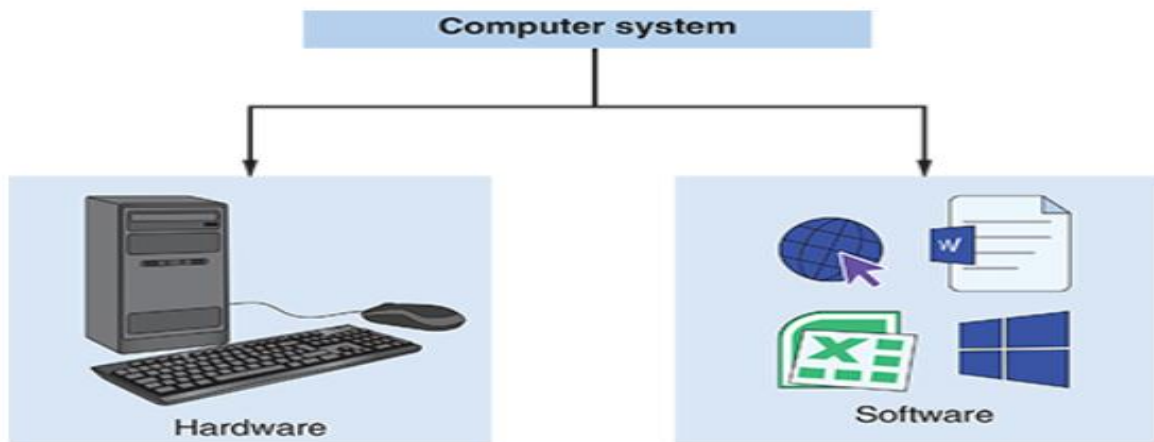
Hardware and software are interdependent components of a computer system.

- Hardware provides the physical resources required to perform operations.
- Software provides the instructions that control how these operations are executed.

Without software, hardware cannot function, and without hardware, software cannot run.

Therefore, both components must work together for a computer system to operate effectively.

Block Representation of Computer System



Importance of Computer Systems

Computer systems have become essential in modern society because they:

- Perform complex calculations quickly
- Store large amounts of data
- Automate repetitive tasks
- Improve efficiency and productivity
- Enable communication through the internet

They are widely used in fields such as:

- Banking
- Education
- Healthcare
- Transportation
- Scientific research
- Artificial Intelligence

A computer system is a powerful combination of hardware and software components working together to perform computational tasks efficiently. Understanding the structure and functioning of computer systems is essential for learning programming languages such as C. Knowledge of these components helps programmers understand how programs interact with the system to solve real-world problems.

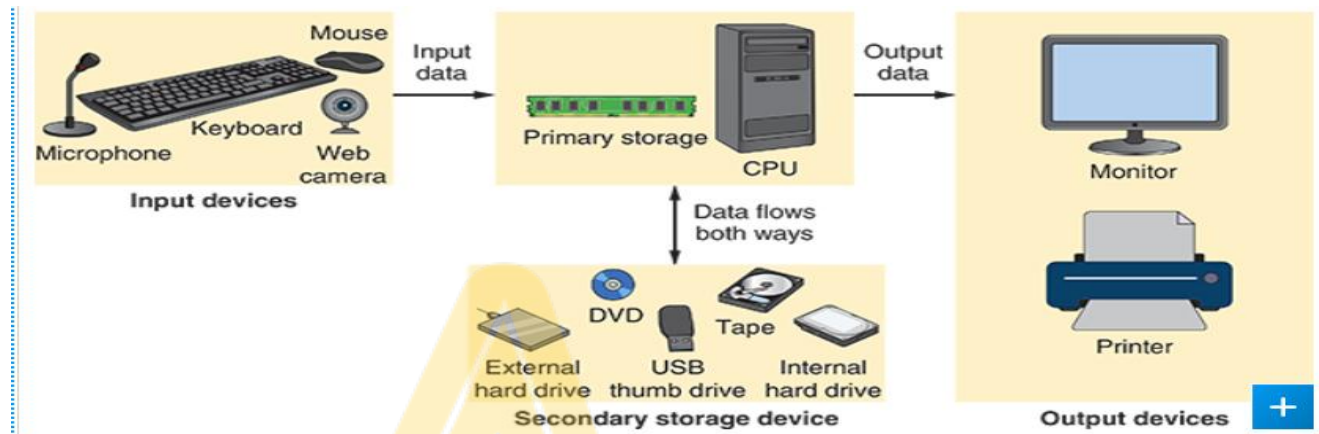
COMPUTER HARDWARE

Computer hardware refers to the **physical components of a computer system that can be seen and touched**. These components work together to perform input, processing, storage, and output operations.

The hardware component of a computer system mainly consists of **five basic parts**:

1. Input Devices
2. Central Processing Unit (CPU)
3. Primary Storage (Main Memory)
4. Output Devices
5. Secondary Storage Devices

These components together determine how a computer system functions and processes data.



Basic Hardware Components of a Computer System

The working of a computer system can be understood using a **data flow model**:

Input → Processing → Storage → Output

- Data enters the computer through **input devices**.
- The **CPU processes** the data.
- The data is temporarily stored in **primary storage**.
- The results are displayed using **output devices**.
- Data and programs are permanently stored in **secondary storage devices**.

1. Input Devices

Input devices are hardware components used to **enter data and instructions into the computer system**.

In simple terms:

Input = Data entering into the computer system

Common Input Devices

Keyboard

The keyboard is the most commonly used input device. It is used to enter text, numbers, and commands into the computer.

Mouse

A mouse is a pointing device used to select items, move the cursor, and interact with graphical interfaces.

Microphone

A microphone allows users to input audio or voice data into the computer.

Web Camera

A web camera captures images and videos and sends them as input to the computer.

Other Input Devices

- Pen or stylus
- Touch screen
- Scanner
- Digital camera

These devices convert human actions into signals that the computer can process.

2. Central Processing Unit (CPU)

The **Central Processing Unit (CPU)** is the most important component of a computer system. It is often referred to as the **brain of the computer** because it controls and performs all operations.

The CPU is responsible for:

- Executing instructions
- Performing arithmetic calculations
- Comparing data
- Controlling the movement of data within the system

Modern computers may contain **one or more CPUs**, allowing faster processing.

Main Functions of CPU

The CPU performs the following operations:

- Arithmetic calculations (addition, subtraction, multiplication, division)
- Logical comparisons
- Data transfer between memory and devices

- Execution of program instructions

3. Primary Storage (Main Memory)

Primary storage is also known as **main memory**. It temporarily stores programs and data that are currently being used by the CPU.

Primary storage mainly consists of:

1. Random Access Memory (RAM)

RAM is a **volatile memory**, meaning the data stored in RAM is lost when the computer is turned off.

RAM temporarily stores:

- Programs currently being executed
- Data required for processing

2. Read Only Memory (ROM)

ROM is **non-volatile memory**, meaning the data stored in ROM is permanent and does not disappear when power is turned off.

ROM usually stores important instructions needed to start the computer system.

Important Feature of Primary Storage

- Stores data temporarily during processing
- Data is erased when the computer is turned off or when a user logs off from a time-sharing system

4. Output Devices

Output devices are used to **display the results produced by the computer after processing**.

In simple terms:

Output = Information coming out of the computer system

Common Output Devices

Monitor

The monitor displays the results visually on the screen.

Printer

A printer produces a printed copy of the results.

Types of Output

Soft Copy

If the output is displayed on a monitor or screen, it is called a **soft copy**.

Hard Copy

If the output is printed on paper using a printer, it is called a **hard copy**.

5. Secondary Storage Devices

Secondary storage is used to **store programs and data permanently**.

Unlike primary storage, data stored in secondary storage **remains even when the computer is turned off**.

Secondary storage devices can act as **both input and output devices** because data can flow in both directions.

Examples of Secondary Storage

- Internal hard drive
- External hard drive
- USB thumb drive
- DVD
- Magnetic tape

Purpose of Secondary Storage

- Long-term storage of data
- Storage of operating systems and application programs
- Backup of important files

In most personal computers, the **hard disk** is the main secondary storage device.

Working of Computer Hardware (Flow of Data)

The functioning of computer hardware can be summarized as follows:

1. Input devices send data to the computer system.
2. The data is stored temporarily in primary memory.
3. The CPU processes the data according to program instructions.
4. The results are displayed through output devices.

5. Programs and data can be stored permanently in secondary storage.

Simple Representation of Hardware Components

Input Devices → CPU → Primary Memory → Output Devices
↓
Secondary Storage

COMPUTER SOFTWARE

Computer software refers to a collection of programs, procedures, and instructions that tell the computer hardware how to perform specific tasks.

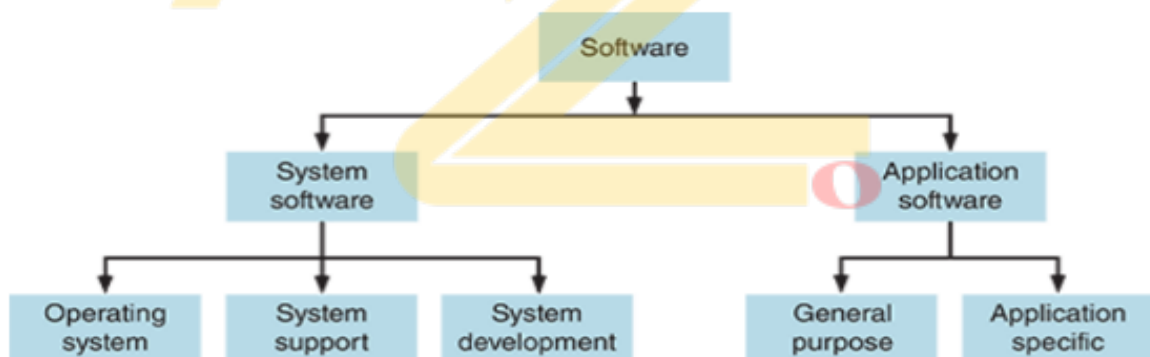
While **hardware represents the physical components of a computer**, software represents the **logical instructions that control the hardware**. Without software, hardware cannot function or perform useful operations.

Software acts as a **bridge between the user and the computer hardware**, enabling users to interact with the system and solve problems efficiently.

Computer software is broadly classified into **two main categories**:

1. System Software
2. Application Software

Types of Computer Software



1. System Software

System software is a set of programs designed to **manage and control the computer hardware and system resources**. It provides a platform on which application software can run.

In other words, system software acts as an **interface between hardware and application programs**.

System software ensures that all components of the computer system work together efficiently.

Components of System Software

System software mainly consists of three parts:

1. Operating System
2. System Support Programs
3. System Development Programs

1. Operating System

An **Operating System (OS)** is the most important type of system software. It manages the hardware resources and controls the execution of programs.

The operating system performs several important functions such as:

- Managing CPU operations
- Managing memory
- Handling input and output devices
- Managing files and storage
- Providing security and user interface

Examples of operating systems include:

- Microsoft Windows
- Linux
- macOS
- Android
- iOS

The operating system acts as a **mediator between the user and the computer hardware**.

2. System Support Programs

System support programs are utility programs that help **maintain, monitor, and manage the computer system**.

These programs assist the operating system in performing system-related tasks.

Examples include:

- Disk management tools
- File management utilities

- Backup and recovery programs
- Antivirus software
- System monitoring tools

These programs ensure the **efficient and secure functioning of the computer system**.

3. System Development Programs

System development programs are tools used by programmers to **develop new software applications**.

These programs help programmers write, test, debug, and maintain computer programs.

Examples include:

- Compilers
- Interpreters
- Assemblers
- Debuggers
- Program editors

For example, a **C compiler** converts C language programs into machine language so that the computer can execute them.

2. Application Software

Application software refers to programs designed to **help users perform specific tasks or solve particular problems**.

Unlike system software, application software focuses directly on **user needs and activities**.

Examples include:

- Word processing
- Spreadsheet calculations
- Internet browsing
- Graphic design
- Data management

Application software is generally divided into **two types**:

1. General Purpose Software

2. Application Specific Software

1. General Purpose Software

General purpose software is designed to perform **common tasks that are required by many users**.

These programs are widely used in everyday activities.

Examples include:

- Microsoft Word (document creation)
- Microsoft Excel (spreadsheet calculations)
- Microsoft PowerPoint (presentations)
- Web browsers (Chrome, Firefox)
- Media players

These applications are flexible and can be used for **multiple purposes**.

2. Application Specific Software

Application specific software is designed to perform **a particular task or solve a specific problem**.

These programs are developed for **a specific organization, industry, or purpose**.

Examples include:

- Banking software
- Hospital management systems
- Railway reservation systems
- Inventory management systems
- Airline booking systems

Such software is customized to meet the **specific requirements of users or organizations**.

Relationship Between System Software and Application Software

Both system software and application software work together to make the computer system functional.

- **System software** controls the computer hardware and provides the platform.
- **Application software** allows users to perform specific tasks.

Without system software, application software cannot run, and without application software, the computer would not be useful to the user.

SYSTEM SOFTWARE

System software refers to a collection of programs designed to **manage and control the computer hardware resources and perform basic information-processing tasks**. It acts as an intermediary between the **computer hardware and application software**, ensuring that the system functions efficiently.

System software is essential because it allows users and application programs to interact with the computer hardware in an organized and controlled manner.

System software is broadly classified into **three categories**:

1. Operating System
2. System Support Software
3. System Development Software

1. Operating System

The **Operating System (OS)** is the most important component of system software. It manages all hardware resources and provides a platform for running application programs.

The operating system acts as a **user interface**, allowing users to interact with the computer system through graphical elements such as:

- Dialog boxes
- Buttons
- Icons
- Text boxes

Through these components, users can provide instructions and interact with programs easily.

Functions of Operating System

The operating system performs several important tasks, including:

1. Resource Management

It manages hardware resources such as CPU, memory, and input/output devices.

2. File Management

It provides access to files and databases stored in the system.

3. User Interface

It allows users to interact with the computer through graphical or command-based interfaces.

4. Communication Management

It interacts with communication systems such as Internet protocols and network connections.

5. System Efficiency

It ensures that the computer system operates smoothly and efficiently while allowing multiple users or programs to access system resources.

Examples of operating systems include:

- Microsoft Windows
- Linux
- macOS
- Android

2. System Support Software

System support software consists of **utility programs and operating services** that assist the operating system in maintaining and managing the computer system.

These programs help improve system performance, maintain system stability, and perform maintenance tasks.

Examples of System Support Programs

System Utilities

Utilities are programs designed to perform maintenance or support operations. Examples include:

- Disk formatting programs
- File sorting programs
- Disk cleanup utilities
- Backup programs

Operating Services

Operating services are programs that help monitor and maintain system performance.

Examples include:

- Performance diagnosis programs
- System statistics monitoring tools
- Resource usage tracking programs

These utilities ensure the computer system runs **efficiently, securely, and reliably**.

3. System Development Software

System development software consists of tools used by programmers to **develop and maintain computer programs**.

These tools help convert high-level programming languages into machine language so that computers can execute programs.

Components of System Development Software

Language Translators

Language translators convert programs written in programming languages into machine language.

Examples include:

- Compilers
- Interpreters
- Assemblers

Debugging Tools

Debugging tools help programmers **identify and correct errors in programs**.

CASE Tools (Computer-Aided Software Engineering)

CASE tools assist programmers in designing, developing, and maintaining software systems.

System development software plays an important role in **software creation and program testing**.

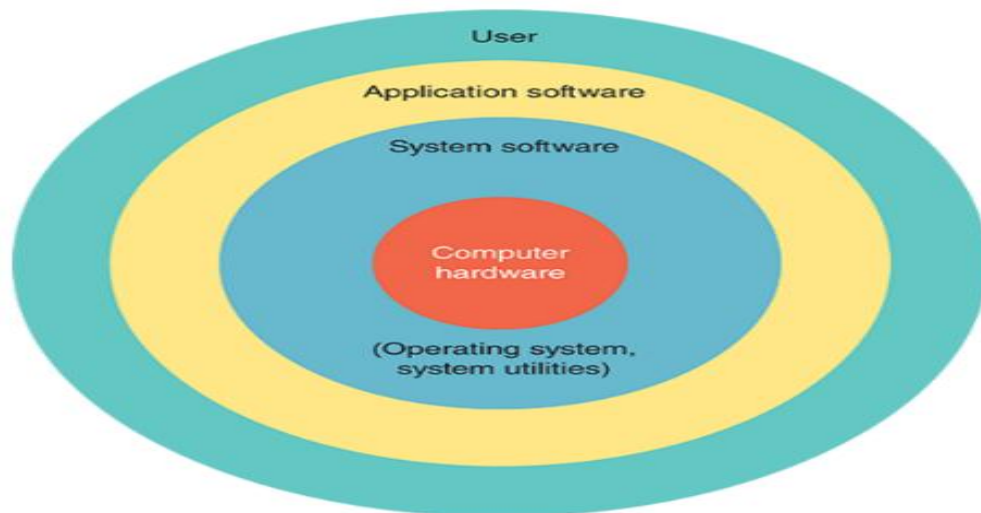
APPLICATION SOFTWARE

Application software refers to programs designed to **help users perform specific tasks or solve particular problems**.

Unlike system software, which manages hardware resources, application software focuses on **user activities and problem solving**.

Application software is divided into **two main categories**:

1. General-Purpose Software
2. Application-Specific Software



1. General-Purpose Software

General-purpose software is developed by software companies and can be used for **multiple applications by many users**.

These programs are designed to **handle common tasks** that are required in everyday computing.

Examples

- Word processors (Microsoft Word)
- Database management systems
- Spreadsheet programs (Microsoft Excel)
- Computer-Aided Design (CAD) systems

These programs are called **general-purpose** because they can solve a **variety of computing problems**.

2. Application-Specific Software

Application-specific software is designed to perform **a particular task for a specific organization or purpose**.

Such software is usually **custom developed** to meet the unique requirements of users.

Examples

- General ledger systems used by accountants
- Material Requirements Planning (MRP) systems used in manufacturing
- Banking software
- Hospital management systems
- Airline reservation systems

These applications are limited to the **specific task for which they were designed** and cannot be used for general computing purposes.

Relationship Between System Software and Application Software

The relationship between system software and application software can be understood as a layered structure.

1. **Computer Hardware** forms the inner core of the system.
2. **System Software** directly interacts with the hardware and controls it.
3. **Application Software** runs on top of system software and performs tasks for users.
4. **Users** interact with the system through application software.

In this structure:

- Application software depends on system software to function.
- System software provides direct access to hardware resources.

However, **system software does not depend on application software**, meaning the system can operate even without application programs.

Role of Programming Languages in Software Development

If users cannot find software that satisfies their needs, developers must **create custom software applications**.

Programming languages are used for this purpose.

One of the important programming languages used to develop software is the **C programming language**, which is widely used for:

- System programming
- Operating systems
- Embedded systems
- Application development

The C language provides powerful features that allow programmers to build efficient and reliable software systems.

COMPUTING ENVIRONMENTS

A **computing environment** refers to the type of system setup in which computers operate and users interact with them. It includes the **hardware, software, and the way computing resources are organized and used**.

In the early days of computing, computers were very large and expensive machines known as **mainframe computers**. These computers were usually located in a **central computing department**, and users accessed them indirectly.

As technology improved, the development of **minicomputers and personal computers** changed the computing environment significantly. Computers became smaller, cheaper, and more accessible, allowing individuals to have computers on their own desks.

Today, computing environments include various types of devices and systems used for personal, professional, and organizational purposes.

PERSONAL COMPUTING ENVIRONMENT

The **personal computing environment** refers to a setup in which an individual uses a computer device for personal or professional tasks.

The development of personal computing began in **1971**, when **Marcian E. Hoff**, working at Intel, designed the first microprocessor called the **Intel 4004**.

This microprocessor combined the essential functions of a **Central Processing Unit (CPU)** onto a single chip. It marked the beginning of modern computing technology and became the ancestor of the advanced processors used in today's computers.



DEVELOPMENT OF PERSONAL COMPUTERS

The invention of microprocessors led to the development of **personal computers (PCs)** in the 1970s. These computers were smaller and more affordable than mainframe computers, allowing individuals and small businesses to use them.

Soon, two major types of personal computers emerged.

1. Microsoft-Based Personal Computers

The first category consisted of computers that ran the **Microsoft Disk Operating System (DOS)** and later the **Microsoft Windows operating system**.

These computers were manufactured by many different companies and became widely popular.

Today, the term **PC (Personal Computer)** is commonly used to refer to computers that run Microsoft operating systems such as Windows.

2. Apple Computers

The second category consisted of computers manufactured by **Apple Inc.**

Apple computers use their own operating systems and hardware designs. They were originally designed as desktop computers but later expanded into various portable devices.

Modern Personal Computing Devices

Over time, personal computing devices have evolved into many forms. Today, users can access computing power through different types of devices.

Some common examples include:

1. Desktop Computers

Desktop computers are traditional personal computers designed to sit on a desk. They usually consist of separate components such as:

- Monitor
- CPU cabinet (tower)
- Keyboard
- Mouse

These systems are commonly used in offices, homes, and educational institutions.

2. Laptops

A laptop is a portable personal computer that integrates the screen, keyboard, processor, and battery into a single device.

Laptops are widely used because they are **lightweight, compact, and easy to carry**.

3. Tablets

Tablets are touch-based portable computing devices that use a touchscreen interface instead of a traditional keyboard and mouse.

They are commonly used for:

- Reading
- Browsing the internet
- Watching videos
- Educational purposes

4. Smartphones

Smartphones are advanced mobile phones that combine **communication capabilities with computing power**.

They allow users to perform various tasks such as:

- Internet browsing
- Email communication
- Multimedia usage
- Running mobile applications

5. All-in-One Computers

All-in-one computers combine the **CPU and monitor into a single device**, reducing the need for separate hardware components.

These systems save space and provide a cleaner, more compact design.

Advantages of Personal Computing Environment

The personal computing environment provides several advantages:

- Easy access to computing resources

- Increased productivity
- Convenience and portability
- Ability to perform multiple tasks
- Availability of powerful software applications

Personal computers are now widely used in:

- Education
- Business
- Research
- Entertainment
- Communication

TIME-SHARING ENVIRONMENT

A **Time-Sharing Environment** is a computing system in which **multiple users share the resources of a single central computer simultaneously**.

In this environment, many users are connected to one powerful computer such as a **mainframe or minicomputer (server)** through terminals. Each user gets a small portion of the computer's processing time, called a **time slice**.

Because the CPU switches rapidly between users, each user feels as if they are working on the computer independently.

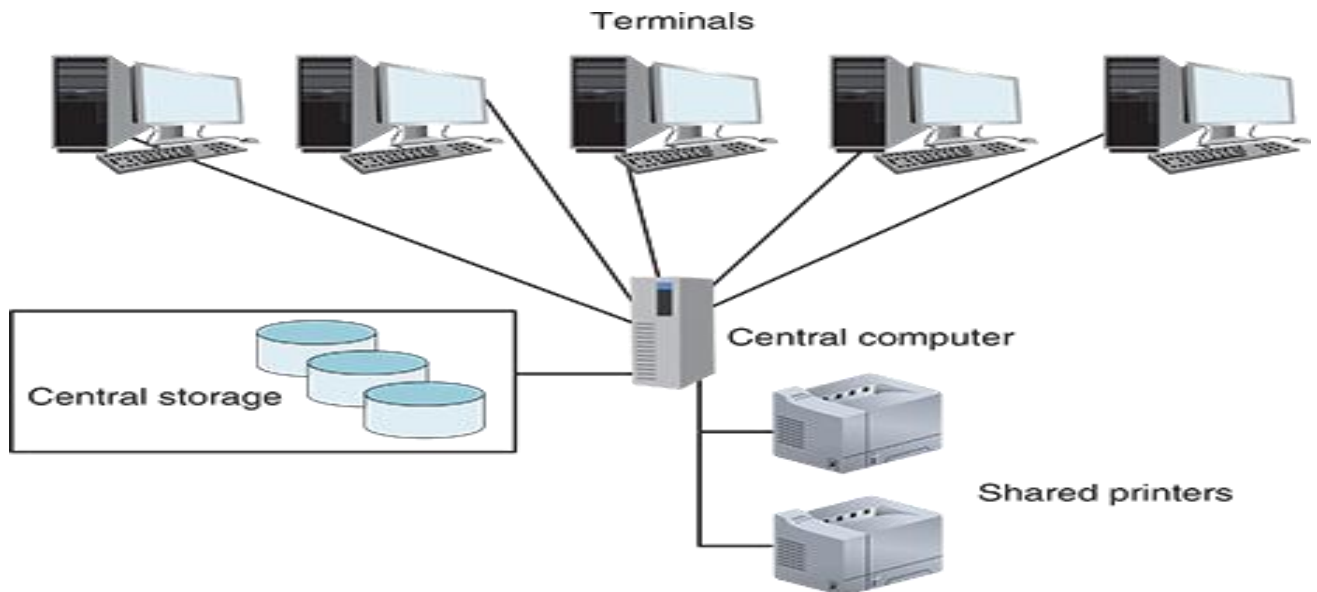
Although time-sharing systems were widely used in the past, today they are less common due to the availability of powerful **personal computers and distributed systems**.

Structure of a Time-Sharing Environment

In a time-sharing environment, the system usually consists of:

1. **Central Computer (Mainframe or Server)**
2. **Multiple Terminals used by users**
3. **Central Storage Devices**
4. **Shared Output Devices such as Printers**

All users access the system through terminals connected to the central computer.



Components of Time-Sharing Environment

1. Central Computer

The **central computer** is the main processing unit of the system. It performs all computations and controls the system resources.

Its responsibilities include:

- Managing CPU time for multiple users
- Controlling input and output devices
- Managing shared data and storage
- Handling user requests simultaneously

Because all processing happens in the **central computer**, it must be **very powerful**.

2. Terminals

Terminals are devices used by users to interact with the central computer.

A terminal typically includes:

- A keyboard for input
- A monitor for output

In older systems, terminals were **non-programmable devices**, meaning they could only send input and display output.

Today, **microcomputers or PCs** can be used to simulate terminals.

3. Central Storage

Central storage refers to storage devices connected to the central computer where all programs and data are stored.

Examples include:

- Hard disks
- Storage servers
- Database systems

All users access the same central storage for retrieving and storing data.

4. Shared Output Devices

Output devices such as **printers** are shared among multiple users in the time-sharing system.

Instead of each user having their own printer, all users send their printing tasks to a **central shared printer**.

Working of Time-Sharing Environment

The working process can be explained as follows:

1. Multiple users connect to the central computer through terminals.
2. Each user submits input or commands.
3. The central computer processes each request by allocating small CPU time intervals.
4. Results are returned to the respective terminals.
5. Output devices such as printers are shared among all users.

Because the CPU switches between users very quickly, it appears that all users are working simultaneously.

Example of Time-Sharing Environment

A common example is a **college computer laboratory** where many students use terminals connected to a central server.

In such a setup:

- Students enter commands from their terminals.
- The central computer performs the processing.

- Output is displayed on their screens or printed using shared printers.

Limitations of Time-Sharing Environment

Although time-sharing systems allow multiple users to work simultaneously, they have some disadvantages:

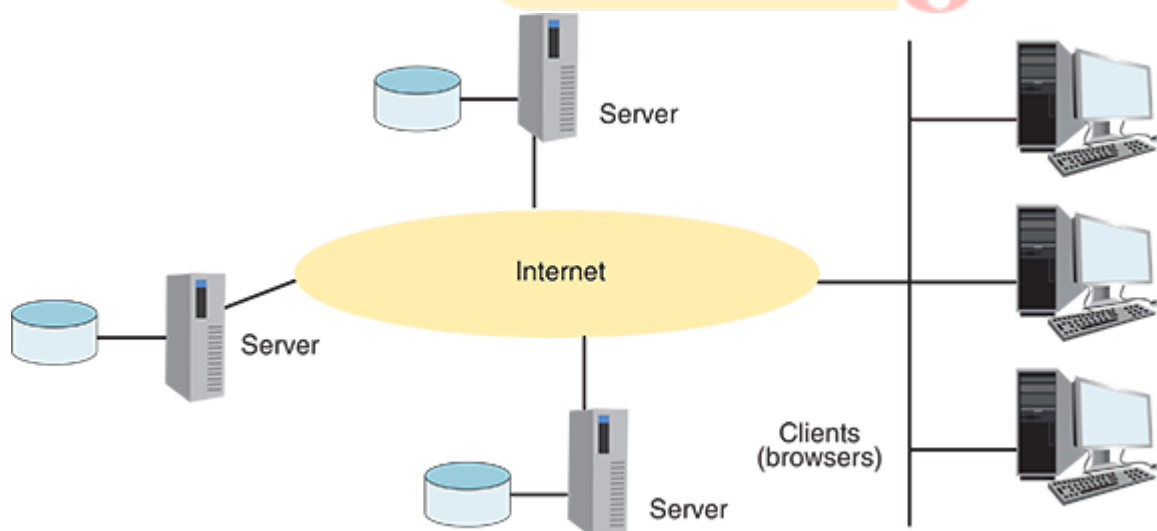
1. **Heavy Workload on Central Computer**
Since all processing occurs in the central computer, it becomes heavily loaded.
2. **Slow Response Time**
When many users access the system simultaneously, the computer may respond slowly.
3. **User Frustration**
Delays in processing may frustrate users when the system becomes too busy.
4. **Dependence on Central Computer**
If the central computer fails, all connected users are affected.

Advantages of Time-Sharing Environment

Despite its limitations, time-sharing systems offer several advantages:

- Multiple users can access a computer simultaneously.
- Efficient use of expensive computing resources.
- Shared access to storage and printers.
- Useful in educational and research environments

DISTRIBUTED COMPUTING ENVIRONMENT



A **Distributed Computing Environment** is a computing system in which **multiple computers (servers and clients) located in different geographical locations work together to perform computing tasks.**

This concept became popular in the **1990s** with the rapid growth of computer networks and the Internet.

In distributed computing, instead of relying on a single powerful computer, the **processing tasks and resources are distributed among multiple interconnected systems.** These systems communicate with each other through a network to perform complex operations.

Components of Distributed Computing Environment

A distributed computing environment typically consists of the following components:

1. Servers
2. Clients
3. Storage Systems
4. Network/Internet

1. Servers

Servers are powerful computers that provide computing resources and services to clients.

In a distributed system, there are **multiple servers** located in different locations. These servers share resources such as:

- CPU processing power
- Memory
- Storage
- Databases

All these resources are **pooled together to provide high processing capability.**

2. Clients

Clients are the computers or devices used by users to access services provided by servers.

Examples include:

- Personal computers
- Laptops

- Smartphones
- Web browsers

Clients send requests to the servers and receive responses through the network.

3. Storage Devices

Distributed systems use storage devices connected to servers to store data.

Examples include:

- Databases
- Hard disks
- Cloud storage systems

These storage systems hold data that can be accessed by servers and then delivered to clients when requested.

4. Network / Internet

The **Internet or computer network** connects servers and clients in distributed computing.

It allows systems located in different geographic areas to communicate and share resources.

The network enables:

- Data transmission
- Resource sharing
- Communication between servers and clients

Working of Distributed Computing Environment

The working process of a distributed system can be described as follows:

1. The client sends a request through the Internet.
2. The request is received by one of the servers in the distributed system.
3. The server processes the request using its CPU, memory, and storage resources.
4. If required, the server may communicate with other servers in the network.
5. The processed result is sent back to the client through the Internet.

This distributed approach allows many tasks to be performed simultaneously.

Example of Distributed Computing

A well-known example of distributed computing is **Amazon's online shopping system**.

Amazon uses **thousands of servers located in multiple data centers** around the world. These servers work together to:

- Store product information
- Process customer orders
- Handle payments
- Deliver web pages to users

This distributed system ensures that the service remains **fast, reliable, and always available**.

Advantages of Distributed Computing

1. **High Processing Power**
Multiple computers work together to perform tasks.
2. **Reliability**
If one server fails, other servers can continue processing.
3. **Scalability**
New servers can be added easily to increase system capacity.
4. **High Availability**
Services remain available to users even if some components fail.
5. **Efficient Resource Sharing**
Resources such as CPU, memory, and storage are shared among systems.

Limitations of Distributed Computing

1. **Complex System Management**
Managing multiple servers and networks can be difficult.
2. **Security Issues**
Data transmitted across networks may be vulnerable to attacks.
3. **Network Dependency**
System performance depends on network speed and reliability

CLOUD COMPUTING

Cloud Computing is a computing environment in which **servers, storage, applications, and other computing resources are provided over the Internet** instead of being stored on a user's personal computer.

Cloud computing began to develop in the **early 2000s** and has become one of the most important technologies in modern computing.

In this environment, data and applications are stored on **remote servers located in different geographic locations**, commonly called **data centers**. Users can access these services using the Internet from anywhere in the world.

Cloud computing allows users to use powerful computing resources **without installing software or maintaining hardware on their local machines**.

Components of Cloud Computing

A cloud computing system mainly **consists** of the following components:

1. Servers
2. Storage Systems
3. Applications
4. Internet / Network
5. Client Devices

1. Servers

Servers are powerful computers located in **data centers** that process requests from users.

These servers handle:

- Application execution
- Data processing
- Resource management

In cloud computing, many servers work together to provide reliable and scalable services.

2. Storage

Cloud systems include large storage facilities where user data is stored.

Examples include:

- Databases

- File storage systems
- Backup storage

Users can store and retrieve data from the cloud anytime through the Internet.

3. Applications

Cloud platforms provide various applications that users can access online.

These applications run on cloud servers instead of the user's device.

Examples include:

- Online document editors
- Email services
- File-sharing systems

4. Internet / Network

The **Internet** connects users to cloud servers.

Users send requests through the Internet, and cloud servers process those requests and return results.

5. Client Devices

Users access cloud services using different devices such as:

- Desktop computers
- Laptops
- Smartphones
- Tablets

These devices act as **clients** that interact with the cloud servers.

Types of Cloud Computing Services

Cloud computing provides services in three main forms:

1. Software as a Service (SaaS)

In **SaaS**, software applications are delivered to users through the Internet.

Users do not need to install the software on their devices.

Example:

- Microsoft **Office 365**
- Google Docs
- Gmail

Users simply open the application in a web browser.

2. Platform as a Service (PaaS)

In **PaaS**, cloud providers supply a platform where developers can **build, test, and deploy applications**.

The platform includes tools, libraries, and development environments.

Example platforms include:

- Google App Engine
- Microsoft Azure

3. Infrastructure as a Service (IaaS)

In **IaaS**, cloud providers offer fundamental computing resources such as:

- Virtual machines
- Storage
- Networking

Users can rent these resources and configure them according to their needs.

Example:

- Amazon Web Services (AWS)

Examples of Cloud Service Providers

Many large technology companies provide cloud computing services, including:

- Amazon (AWS)
- Google Cloud
- Microsoft Azure
- IBM Cloud
- Oracle Cloud

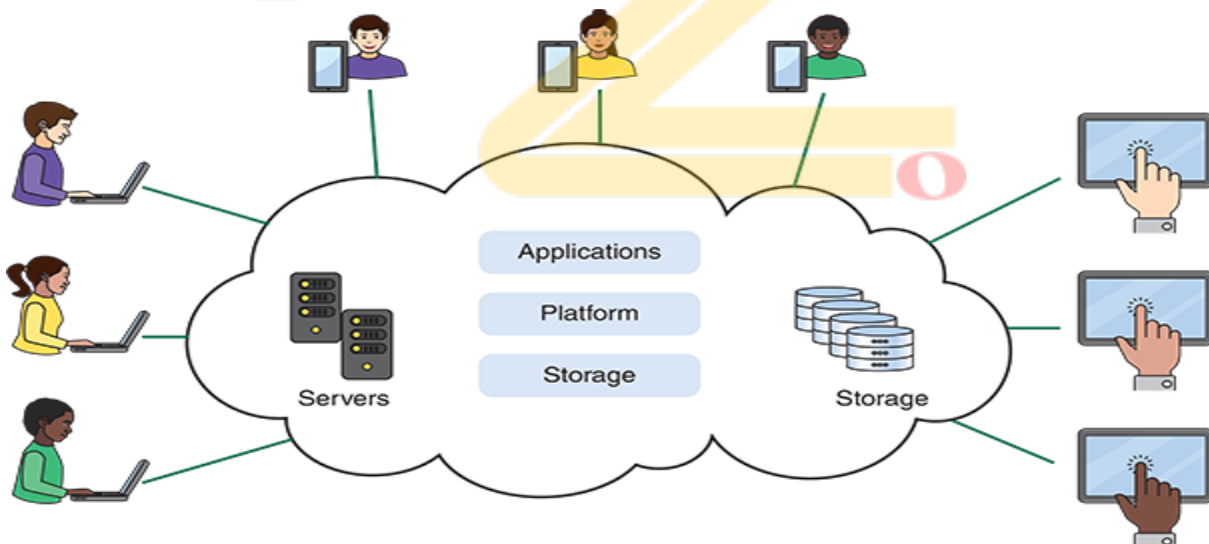
These companies operate **large data centers worldwide**.

Advantages of Cloud Computing

1. **Cost Efficiency**
Users do not need to buy expensive hardware.
2. **Scalability**
Resources can be increased or decreased based on demand.
3. **Accessibility**
Users can access services from anywhere with Internet access.
4. **Data Backup and Security**
Cloud providers maintain secure backups of data.
5. **Reduced Maintenance**
Users do not need to maintain software or hardware systems.

Limitations of Cloud Computing

1. **Internet Dependency**
Cloud services require a stable Internet connection.
2. **Security Concerns**
Data stored on remote servers may be vulnerable to cyber attacks.
3. **Limited Control**
Users depend on cloud providers for infrastructure management.



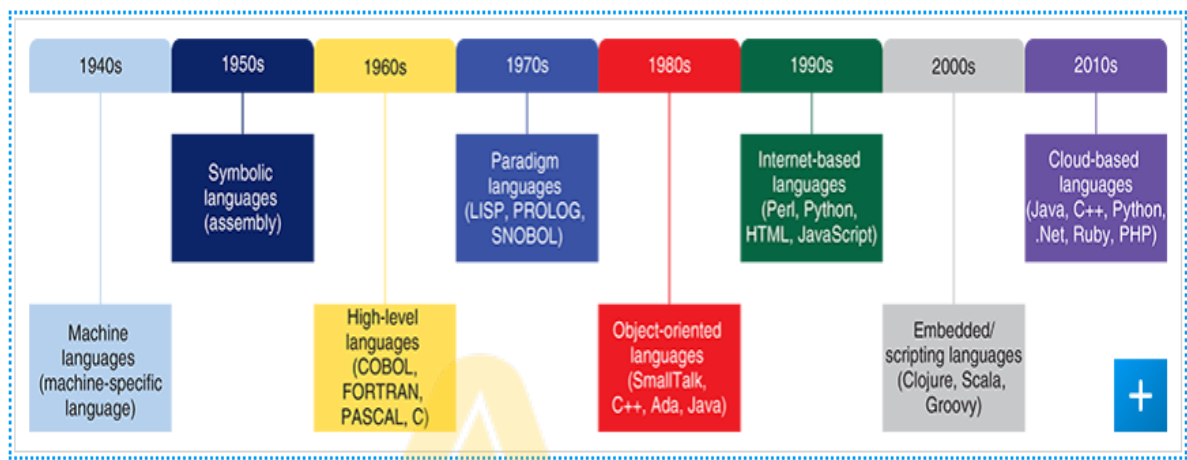
COMPUTER LANGUAGES

A **computer language** is a formal language used to write instructions that a computer can understand and execute.

Computers cannot understand human language directly. Therefore, programmers use **programming languages** to communicate with computers and instruct them to perform specific tasks.

Over time, computer languages have **evolved significantly**, moving from simple machine-level instructions to advanced high-level languages that are easier for humans to understand.

This evolution has made programming **more efficient, flexible, and user-friendly**.



EVOLUTION OF COMPUTER LANGUAGES

Computer languages have evolved through several stages over different decades.

1. Machine Languages (1940s)

Machine language was the **first generation programming language**.

It consists of **binary digits (0s and 1s)** that are directly understood by the computer hardware.

Example:

10110000 01100001

Characteristics

- Machine-specific
- Difficult for humans to read and write
- Requires detailed knowledge of hardware
- Error-prone and time-consuming

Advantage

- Executes very fast because it directly interacts with the processor.

Disadvantage

- Very difficult to program and debug.

2. Assembly Languages (1950s)

Assembly language was developed to make programming easier than machine language.

Instead of binary codes, it uses **symbolic instructions called mnemonics**.

Example:

```
ADD A, B  
MOV R1, R2
```

Characteristics

- Uses symbolic names instead of binary numbers
- Requires an **assembler** to convert the code into machine language
- Still hardware-dependent

Advantage

- Easier to understand than machine language.

Disadvantage

- Still complex and machine-specific.

3. High-Level Languages (1960s)

High-level languages were developed to make programming **more user-friendly and independent of hardware**.

These languages use **English-like statements**, making them easier to read, write, and maintain.

Examples

- COBOL
- FORTRAN
- Pascal
- C

Characteristics

- Machine independent
- Easier to program
- Requires a **compiler or interpreter** to translate the program into machine language

High-level languages significantly improved **program development speed and portability**.

4. Paradigm Languages (1970s)

During the 1970s, programming languages began focusing on **different programming paradigms or approaches**.

These languages introduced new ways of solving problems using logic and artificial intelligence techniques.

Examples

- LISP
- PROLOG
- SNOBOL

These languages were widely used in **artificial intelligence research and symbolic processing**.

5. Object-Oriented Languages (1980s)

Object-Oriented Programming (OOP) languages were developed to organize programs using **objects and classes**.

These languages support concepts such as:

- Encapsulation
- Inheritance
- Polymorphism

Examples

- Smalltalk
- C++
- Java
- Ada

Object-oriented languages help developers build **large and complex software systems efficiently**.

6. Internet-Based Languages (1990s)

With the rapid growth of the **Internet and web technologies**, new programming languages were developed for web applications.

Examples

- Perl

- Python
- HTML
- JavaScript

These languages are used to develop:

- Websites
- Web applications
- Online services

7. Embedded and Scripting Languages (2000s)

In the 2000s, scripting languages became popular for **automation, web development, and system integration**.

Examples

- Clojure
- Scala
- Groovy

These languages are often used for:

- Application scripting
- Data processing
- System automation

8. Cloud-Based Languages (2010s)

With the rise of **cloud computing**, programming languages evolved to support distributed and cloud-based systems.

Examples

- Java
- C++
- Python
- .NET
- Ruby

- PHP

These languages are widely used in:

- Cloud applications
- Web services
- Large-scale distributed systems

Timeline of Computer Language Evolution

1940s → Machine Language

1950s → Assembly Language

1960s → High-Level Languages

1970s → Paradigm Languages

1980s → Object-Oriented Languages

1990s → Internet-Based Languages

2000s → Embedded / Scripting Languages

2010s → Cloud-Based Languages

Importance of Programming Languages

Programming languages are important because they:

- Enable communication between humans and computers
- Simplify program development
- Improve software reliability and maintainability
- Allow development of complex applications
- Support modern technologies such as AI, cloud computing, and web development

MACHINE LANGUAGES

In the earliest days of computing, the only programming language available was **machine language**.

Machine language consists of **binary instructions made up of 0s and 1s** that are directly understood by the computer hardware.

Each computer system has **its own machine language**, which depends on the processor architecture.

1	00000000	00000100	0000000000000000
2	01011110	00001100	11000010 0000000000000010
3	11101111	00010110	00000000000000101
4	11101111	10011110	00000000000001011
5	11111000	10101101	11011111 0000000000010010
6	01100010	11011111	0000000000010101
7	11101111	00000010	11111011 0000000000010111
8	11110100	10101101	11011111 0000000000011110
9	00000011	10100010	11011111 0000000000100001
10	11101111	00000010	11111011 0000000000100100
11	01111110	11110100	10101101
12	11111000	10101110	11000101 0000000000101011
13	00000110	10100010	11111011 0000000000110001
14	11101111	00000010	11111011 0000000000110100
15	01010000	11010100	0000000000111011
16	00000100	0000000000111101	

Why Machine Language Uses 0s and 1s

The internal circuits of a computer consist of **switches, transistors, and electronic components** that have two states:

- **ON → represented by 1**
- **OFF → represented by 0**

Because of this binary behavior, machine instructions are written as **streams of binary numbers**.

Example of machine instruction:

10110000 01100001

These binary instructions tell the processor what operations to perform.

Characteristics of Machine Language

- Written in **binary format (0s and 1s)**
- **Machine dependent**
- Executed directly by the CPU
- Very difficult for humans to read or write

Advantages

1. Directly executed by the computer.
2. Very **fast execution**.
3. No translator required.

Disadvantages

1. Very difficult to write and understand.

2. Programs are **machine-specific**.
3. Debugging and modification are extremely difficult.

SYMBOLIC LANGUAGES (ASSEMBLY LANGUAGES)

Machine language was extremely difficult to use, so programmers needed a simpler method.

In the **early 1950s**, **Admiral Grace Hopper** introduced the concept of a program that converts readable instructions into machine language.

These languages used **symbols (mnemonics)** instead of binary numbers.

Because they use symbols, they are called **symbolic languages**, also known as **assembly languages**.

```
1      entry    main,<r2>
2      subl2    #12, sp
3      jsb      C$MAIN_ARGS
4      movab    $CHAR_STRING_CON
5      pushal   -8(fp)
6      pushal   (r2)
7      calls    #2,SCANF
8      pushal   -12(fp)
9      pushal   3(r2)
10     calls    #2,SCANF
11     mull3     -8(fp),-12(fp),-
12     pushal   6(r2)
13     calls    #2,PRINTF
14     clrl     r0
15     ret
```

Example of Assembly Instructions

Instead of binary:

10110000

Assembly language uses mnemonics like:

MOV A, B

ADD A, B

These mnemonics represent machine instructions in a more readable form.

Structure of Assembly Programs

Assembly language programs are typically written in **three columns**:

1. **Label** – identifies instruction location
2. **Operator (Mnemonic)** – instruction name

3. **Operand** – data or memory location

Example:

```
START  MOV A,B
      ADD A,C
      STORE RESULT
```

Assembler

A computer cannot directly understand assembly language.

Therefore, a special program called an **assembler** converts assembly code into machine language.

Assembly Language → Assembler → Machine Language

Advantages

1. Easier to understand than machine language.
2. Faster execution than high-level languages.
3. Allows direct control of hardware.

Disadvantages

1. Still **machine dependent**.
2. Programming is time-consuming.
3. Difficult for large software development.

HIGH-LEVEL LANGUAGES

Although assembly language improved programming, it still required programmers to write instructions specific to a particular machine.

To make programming easier and focus on solving problems instead of hardware details, **high-level languages** were developed.

High-level languages use **English-like statements**, making them easier to read, write, and maintain.

```
1  /* This program reads two integers from the keyboard
2     and prints their product.
3     Written by:
4     Date:
5  */
6  #include <stdio.h>
7
8  int main (void)
9  {
10 // Local Definitions
11     int number1;
12     int number2;
13     int result;
14
15 // Statements
16     scanf ("%d", &number1);
17     scanf ("%d", &number2);
18     result = number1 * number2;
19     printf ("%d", result);
20     return 0;
21 } // main
```

Key Features of High-Level Languages

- Machine independent
- Easy to learn and use
- Focus on problem-solving instead of hardware details
- Portable across different computers

However, computers cannot directly understand high-level languages.

Therefore, these programs must be **translated into machine language** using a compiler or interpreter.

Compilation Process

High-Level Program



Compiler



Machine Language



Execution

Early High-Level Languages

FORTRAN (1957)

Developed by **John Backus and IBM team**.

FORTRAN stands for:

FORmula TRANslation

It is widely used in:

- Scientific computing
- Engineering calculations
- Mathematical modeling

COBOL

COBOL stands for:

COmmon Business-Oriented Language

Used mainly for:

- Business applications
- Financial systems
- Banking software

Admiral **Grace Hopper** played an important role in its development.

C Language

C is a powerful high-level programming language used for:

- System software
- Operating systems
- Application development

Example of a C program:

```
#include <stdio.h>
```

```
int main()
{
    int number1, number2, result;

    scanf("%d", &number1);
    scanf("%d", &number2);
```

```
result = number1 * number2;

printf("%d", result);

return 0;
}
```

This program reads two numbers and prints their product.

Comparison of Programming Languages

Feature	Machine Language	Assembly Language	High-Level Language
Representation	Binary (0s and 1s)	Mnemonics	English-like syntax
Ease of Use	Very difficult	Moderate	Easy
Portability	Machine dependent	Machine dependent	Machine independent
Translator	Not required	Assembler	Compiler / Interpreter

CREATING AND RUNNING PROGRAMS

Computers can understand only **machine language (binary instructions)**. Therefore, a program written in a high-level language such as **C** must be converted into machine language before it can run.

The process of converting a C program into an **executable** program involves several steps. During program development, these steps are often repeated to **correct errors and improve the program**.

The programmer performs four major steps to create and run a C program:

1. Writing and editing the program
2. Compiling the program
3. Linking the program
4. Executing the program

Steps in Building a C Program

Write Program → Compile → Link → Execute → Output

WRITING AND EDITING PROGRAMS

The first step in program development is **writing the program**.

Programs are written using a software tool called a **text editor**.

A text editor allows programmers to:

- Write program code
- Modify existing code
- Save the program file

Examples of text editors include:

- Notepad
- Visual Studio Code
- Code::Blocks
- Dev C++

These editors often provide helpful **features** such as:

- Syntax highlighting (coloring code elements)
- Search and replace functions
- Copy and paste operations
- Automatic indentation and formatting

After writing the program, it is saved as a **source file**.

For C programs, the file usually has the extension:

filename.c

Example:

program.c

This file contains the **source code written by the programmer**.

COMPILING PROGRAMS

After writing the program, the next step is **compilation**.

A **compiler** converts the C source code into machine language.

The C compiler performs two major tasks:

1. Preprocessing
2. Translation

Preprocessor

The **preprocessor** prepares the program before compilation.

It processes special instructions called **preprocessor directives**.

Example:

```
#include <stdio.h>
```

The preprocessor performs tasks such as:

- Including library files
- Replacing macros
- Removing comments
- Preparing the program for compilation

The output of the preprocessing stage is called the **translation unit**.

Translator

After preprocessing, the **translator** converts the prepared code into machine language.

The result is called an **object file**.

Example output:

program.obj

However, this object file is **not yet executable** because it does not include required library functions.

LINKING PROGRAMS

The next step is **linking**.

Many C programs use predefined functions such as:

- printf()
- scanf()
- Mathematical functions

These functions are stored in **libraries**.

The **linker** combines:

- Object file produced by the compiler
- Required library functions
- Other program modules

After linking, the final **executable file** is created.

Example:

program.exe

Now the program is ready to run.

EXECUTING PROGRAMS

The final step is **executing the program**.

Execution means **running the program to produce output**.

When a program is executed:

1. The operating system loads the program into **primary memory (RAM)**.
2. A special system program called the **loader** performs this loading process.
3. The CPU begins executing the instructions.

During execution, the program:

- Reads input from the user or files
- Processes the data
- Produces output

The output may be displayed on:

- Monitor
- Printer
- File

After the program finishes its task, control **returns to the operating system**, and the program is removed from memory.

SYSTEM DEVELOPMENT

System development is the process of designing, building, testing, and maintaining software systems.

Developing a program is not just about writing code. It requires a **systematic and well-planned approach** to ensure that the program is:

- Efficient
- Reliable
- Error-free
- Easy to maintain

If programmers carefully design programs using structured development techniques, the final system will be more **successful and easier to manage**.

Many programmers believe that **every program contains at least one bug** waiting to cause problems under unusual conditions. A famous example is the **Y2K bug**, which caused many programs to fail on January 1, 2000.

System Development Life Cycle (SDLC)

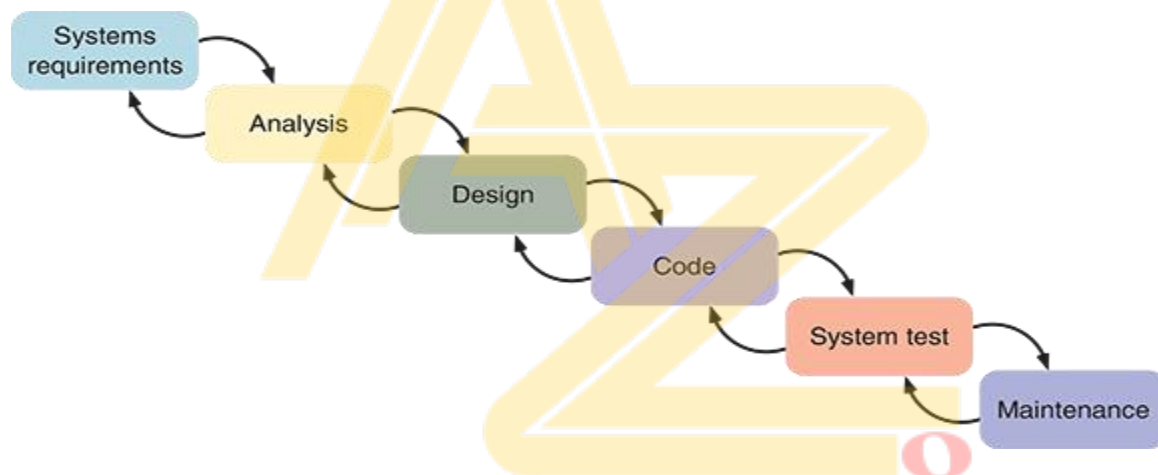
The **System Development Life Cycle (SDLC)** is a structured process used to develop software systems.

It consists of several phases that guide the development process from the initial idea to system maintenance.

One popular SDLC model is the **Waterfall Model**.

Waterfall Model

The **Waterfall Model** is a sequential development process where each phase must be completed before moving to the next phase.



Phases of the Waterfall Model

System Requirements



Analysis



Design



Coding



System Testing



Maintenance

1. System Requirements

In this phase, the **systems analyst identifies and defines the requirements** of the system.

The requirements describe:

- What the system should do
- What problems it should solve
- What results it should produce

These requirements are usually written in **simple language that users can understand**.

2. Analysis

In the **analysis phase**, the analyst studies different possible solutions and determines the best approach for developing the system.

This phase focuses on:

- System feasibility
- Alternative solutions
- System constraints

3. Design

In the **design phase**, the overall structure of the system is planned.

This includes:

- Designing program modules
- Designing data structures
- Designing files and databases
- Determining how different programs interact

4. Coding

The **coding phase** is where the actual program is written using a programming language such as C.

Programmers translate the design into program instructions.

5. System Testing

After coding, the entire system is tested to ensure that all programs work correctly together.

Testing helps identify:

- Logical errors
- Runtime errors
- System integration problems

6. Maintenance

After the system is deployed and used in real-world environments, maintenance is required.

Maintenance includes:

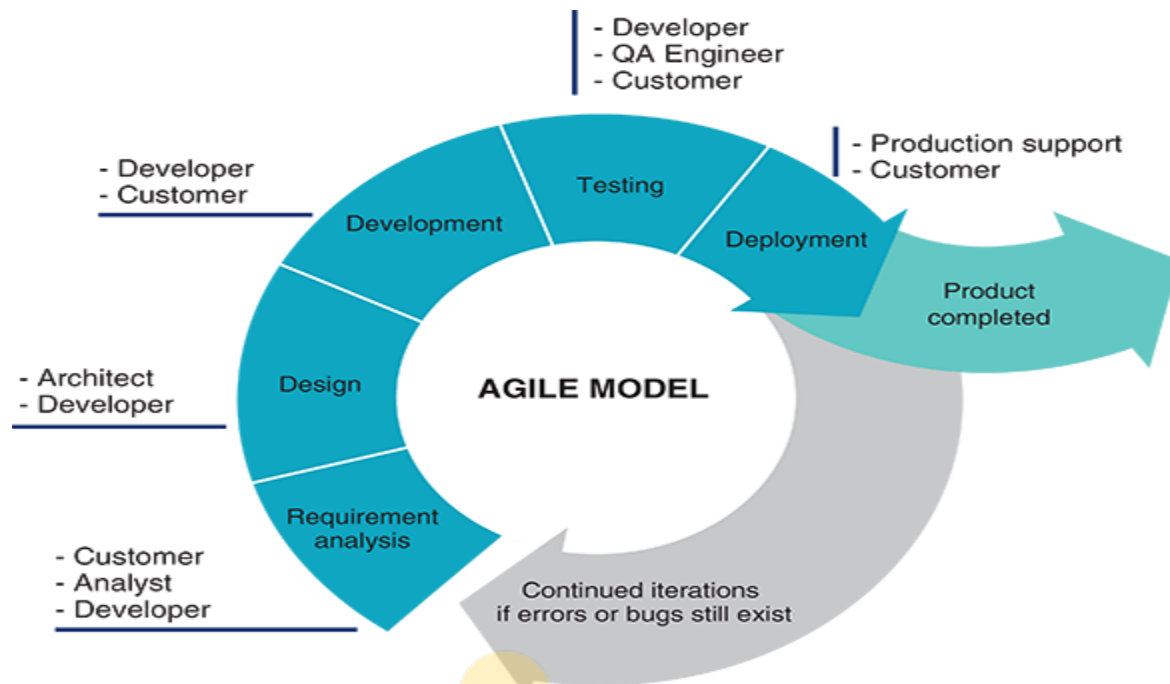
- Fixing bugs
- Improving system performance
- Updating software features

Agile Development Model

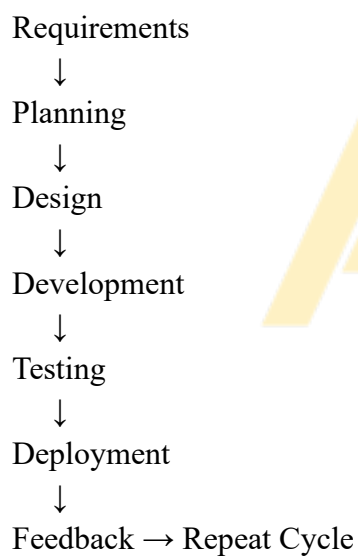
Modern software development often uses the **Agile methodology** instead of the traditional waterfall model.

Agile focuses on:

- **Iterative development**
- **Frequent customer feedback**
- **Short development cycles called sprints**



Agile Development Cycle



Advantages of Agile

- Faster development
- Adaptability to changes
- Better collaboration among team members
- Continuous customer involvement

Program Development

Program development refers to the process of creating a program to solve a specific problem.

The major steps involved in program development are:

1. Understanding the problem
2. Developing a solution
3. Writing the program
4. Testing the program

1. Understanding the Problem

The first step in program development is to **clearly understand the problem requirements**.

Programmers must carefully analyze the problem and ask questions such as:

- What is the expected input?
- What output is required?
- How will the output be used?
- Are there special conditions or constraints?

Clarifying these questions helps prevent errors later in development.

2. Developing the Solution

After understanding the problem, the next step is to **design a solution**.

Three tools commonly used for designing programs are:

- Structure charts
- Pseudocode
- Flowcharts

Usually programmers use **structure charts with either pseudocode or flowcharts**.

Structure Chart

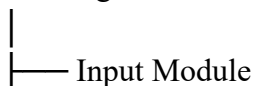
A **structure chart** (also called a hierarchy chart) shows how a program is divided into smaller modules.

It represents the **overall organization of the program**.

Each module performs a specific task.

Example:

Main Program





Structure charts help programmers understand the **functional relationship between different modules**.

Pseudocode

Pseudocode is a program design tool that uses a mixture of **English statements and programming logic** to describe the steps of an algorithm.

Example:

```
Start
Input length
Input width
Area = length × width
Print area
End
```

Pseudocode helps programmers design the logic before writing actual code.

Flowchart

A **flowchart** is a graphical representation of program logic using standard symbols.

It shows the **sequence of steps and flow of data** in a program.

Example flowchart structure:

```
Start
↓
Input Data
↓
Process Data
↓
Display Output
↓
End
```

Flowcharts make it easier to visualize the program structure.

3. Writing the Program

After designing the solution, the programmer writes the actual program.

This process follows **top-down implementation**, where the program is written starting from the main module and then broken into smaller functions.

In simple programs, there may be only **one module (main function)**.

4. Testing the Program

After writing the program, it must be tested to ensure it works correctly.

Testing helps detect errors known as **bugs**.

Two major types of testing are:

Black Box Testing

In **black box testing**, the tester does not know the internal structure of the program.

The program is tested based only on:

- Input
- Output
- Requirements

The tester checks whether the program produces correct results.

White Box Testing

In **white box testing**, the tester understands the internal structure of the program.

The programmer checks:

- Every line of code
- All logical conditions
- All loops and branches

The goal is to ensure that **every part of the program works correctly**.

Important Testing Guidelines

Good testing should ensure that:

- Every line of code is executed at least once.
- Every condition executes both **true and false cases**.

- Boundary values are tested.
- Error conditions are tested.

Large software systems may require **20 or more test cases** to ensure reliability.

OVERVIEW OF THE C PROGRAMMING LANGUAGE

C is one of the most influential programming languages in the history of computing. It has played a major role in the development of modern software systems and many modern programming languages.

The chapter is especially useful for **beginners who are new to C programming**, because it introduces the background and evolution of the language before learning its syntax and programming concepts.

A BRIEF HISTORY OF C

The **C programming language** was invented and first implemented by **Dennis Ritchie** in the early **1970s** at **Bell Laboratories**.

It was developed for use on a **DEC PDP-11 computer system** that ran the **UNIX operating system**. In fact, one of the most important achievements of C was that it was used to **rewrite the UNIX operating system**, making UNIX portable across different computer architectures.

The development of C did not occur suddenly. Instead, it evolved from earlier programming languages.

Evolution of the C Language

The historical development of C can be traced through the following sequence:

BCPL → B → C

BCPL (Basic Combined Programming Language)

The earliest language in this sequence was **BCPL**, which was developed by **Martin Richards** during the 1960s.

Characteristics of BCPL:

- Designed primarily for system programming
- Simple language structure
- Influenced many later programming languages

However, BCPL lacked several advanced features needed for efficient system programming.

B Language

Later, **Ken Thompson** developed a programming language called **B** at Bell Laboratories.

The B language was based on BCPL and was used in the early development of the **UNIX operating system**.

However, B also had some limitations:

- Limited data types
- Less efficient memory handling
- Not powerful enough for complex system programming

Development of C

To overcome the limitations of the B language, **Dennis Ritchie enhanced B and created the C programming language** in the early 1970s.

C introduced several important improvements:

- New data types such as int, char, and float
- Structured programming features
- Efficient memory handling
- Better system programming capabilities

Because of these features, C quickly became the preferred language for **system software development**.

Early Standardization of C

For many years, the **version of C that came with the UNIX operating system** became the unofficial or **de facto standard**.

The first widely recognized description of the C language appeared in the famous book:

“The C Programming Language”
by **Brian Kernighan and Dennis Ritchie**
published in **1978**.

This book became the primary reference for C programmers and is often called **K&R C**.

ANSI Standardization of C

As C grew in popularity, different versions of the language began to appear, which created compatibility issues.

To solve this problem, in **1983 the American National Standards Institute (ANSI)** formed a committee to develop an official standard for the C language.

The standardization process took approximately **six years**, which was much longer than originally expected.

Finally, the **ANSI C standard** was officially adopted in **December 1989**.

The first official copies of the standard became available in **early 1990**.

ISO Adoption of the C Standard

After ANSI adopted the C standard, the **International Organization for Standardization (ISO)** also adopted the same standard.

As a result, the language standard became known as:

ANSI/ISO Standard C

The version of C defined by this standard is commonly referred to as:

C89

Amendment to the C Standard

In **1995**, an amendment to the C standard was introduced.

This amendment added several improvements, including:

- Additional library functions
- Improvements to the standard library

The **C89 standard along with this amendment** later became the foundation for **C++**, defining the subset of C used in the C++ language.

Development of the C99 Standard

During the **1990s**, most programmers focused on the development of **C++**.

However, work on improving the C language continued quietly.

Eventually, a new version of the C standard was released in **1999**, known as:

C99

The C99 standard retained almost all features of C89 while introducing several improvements.

Features Introduced in C99

The C99 standard focused mainly on two areas:

1. Addition of Numeric Libraries

C99 introduced new mathematical and numeric libraries that improved the language's capabilities in scientific and numerical computing.

2. New Language Features

C99 also introduced several new programming features, including:

- **Variable-length arrays (VLAs)**

These allow arrays whose size is determined during program execution.

Example:

```
int n;  
scanf("%d",&n);  
int arr[n];
```

- **restrict pointer qualifier**

This improves program optimization by informing the compiler about pointer usage.

These features helped maintain C's position as a **powerful and modern programming language**.

Importance of the C89 Standard

Despite the introduction of C99, the **C89 standard remains the most widely supported version of C**.

Reasons include:

- Supported by almost all C compilers
- Provides maximum portability
- Forms the basis for the **C++ programming language**

Therefore, many textbooks and programming courses focus primarily on **C89 features**.

C AS A MIDDLE-LEVEL LANGUAGE

The C programming language is often described as a **middle-level programming language**. This classification does not imply that C is inferior to high-level languages such as BASIC or Pascal, nor does it mean that C has the complexity of low-level languages such as assembly language.

Instead, C is called a middle-level language because it **combines the powerful features of high-level languages with the flexibility and control of low-level languages**.

Because of this unique combination, C allows programmers to write programs that are both **efficient and portable**, making it suitable for a wide range of applications, from system software to application programs.

Position of C Among Programming Languages

Programming languages can generally be divided into three categories:

High-Level Languages Middle-Level Languages Low-Level Languages

Ada	Java	Macro-Assembler
-----	------	-----------------

Modula-2	C++	Assembler
----------	-----	-----------

Pascal	C	
--------	---	--

COBOL	FORTH	
-------	-------	--

FORTRAN

BASIC

From this classification, we can see that **C occupies the middle position between high-level and low-level languages.**

Why C is Called a Middle-Level Language

C is considered a middle-level language because it provides features of both **high-level languages and low-level languages.**

High-Level Features of C

Like high-level languages, C supports several **advanced** programming concepts that make programming easier.

1. Structured Programming

C supports structured programming techniques using control structures such as:

- if
- switch
- for
- while
- do-while

These structures help programmers write programs in a logical and organized manner.

Example:

```
if(x > 10)
{
    printf("Number is greater than 10");
}
```

2. Data Types

High-level languages support the concept of **data types**, which define the type of values that variables can store.

Common C data types include:

- int – integer values
- char – characters
- float – decimal numbers
- double – double precision numbers

Example:

```
int age = 20;
char grade = 'A';
float salary = 1500.50;
```

These data types make programs easier to understand and manage.

3. Portability

One of the most important features of C is **portability**.

Portability means that programs written in C can be easily adapted to run on different computer systems or operating systems with minimal modifications.

For example, a program written for the **DOS operating system** can be modified slightly to run on **Windows or Linux systems**.

This feature makes C extremely useful for developing software that must run on multiple platforms.

Low-Level Features of C

In addition to high-level features, C also provides several capabilities normally associated with low-level languages.

1. Direct Memory Access

C allows programmers to access and manipulate memory directly using **pointers**.

Pointers store the **memory address of variables**, allowing programs to interact directly with hardware memory.

Example:

```
int x = 10;  
int *p;  
p = &x;
```

Here, p stores the memory address of the variable x.

2. Bit-Level Manipulation

C allows manipulation of **bits and bytes**, which are the fundamental units used by computers.

Operations such as bit shifting and bitwise operations are possible.

Example:

```
int x = 5;  
int y = x << 1;
```

These operations are commonly used in **system programming and embedded systems**.

3. Address Manipulation

C allows manipulation of memory addresses, which enables programmers to control how data is stored and accessed in memory.

This capability is particularly important when developing:

- Operating systems
- Device drivers
- Embedded systems

Flexibility of Data Types

Although C supports data types, it is **not a strongly typed language** like Pascal or Ada.

A strongly typed language strictly enforces data type rules.

However, C allows flexible type conversions between different data types.

Example”

```
int a = 65;  
char b;  
b = a;
```

Here, an integer value is converted into a character.

This flexibility gives programmers more control but also requires careful programming.

Lack of Runtime Error Checking

Another distinctive feature of C is that it performs **very little runtime error checking**.

For example, C does not automatically check whether an array index exceeds its limit.

Example:

```
int arr[5];  
arr[10] = 50;
```

This may cause a program error because the array index exceeds the allocated memory.

In C, ensuring program correctness is primarily the **responsibility of the programmer**.

Flexible Parameter Passing

In many high-level languages, the type of an argument must exactly match the type of the parameter.

However, C does not require strict type compatibility.

C allows arguments of different types as long as they can be **converted to the required parameter type**.

Example:

```
void display(int x)  
{  
    printf("%d", x);  
}
```

```
display('A');
```

Here a character argument is automatically converted to an integer.

Small Number of Keywords

Another important feature of C is its **small number of keywords**.

Keywords are reserved words that form the core commands of the language.

- C89 defines **32 keywords**

- C99 adds **5 additional keywords**

Compared to other languages, this number is relatively small.

For example, some versions of **BASIC contain more than 100 keywords**.

Because C has fewer keywords, the language is **compact, efficient, and easier to learn**.

Advantages of C as a Middle-Level Language

Because of its middle-level characteristics, C provides several advantages:

- Combines efficiency of low-level languages with simplicity of high-level languages
- Provides direct hardware access
- Supports structured programming
- Produces fast and efficient programs
- Allows development of both system and application software

Applications of C

Due to its flexibility and efficiency, C is widely used in:

- Operating systems (UNIX, Linux)
- Embedded systems
- Compilers
- Device drivers
- System utilities
- Application software

C AS A STRUCTURED LANGUAGE

The **C programming language** is commonly referred to as a **structured programming language**. Structured programming is a programming paradigm that organizes programs into **logical blocks or modules** to improve readability, reliability, and maintainability.

Although C shares many characteristics with **block-structured languages**, technically it is not a pure block-structured language. However, because of its ability to divide programs into logical sections using functions and code blocks, C is still considered a structured language.

Structured programming helps programmers design programs in a **clear, organized, and modular way**, making it easier to understand, debug, and maintain large programs.

Difference Between Structured and Block-Structured Languages

A **block-structured language** allows functions or procedures to be defined **inside other functions or procedures**.

Examples of block-structured languages include:

- ALGOL
- Pascal
- Modula-2

In these languages, functions can be nested inside other functions.

However, in C, functions **cannot be defined inside other functions**. Each function must be defined separately.

Because of this restriction, C cannot technically be called a block-structured language. Instead, it is referred to as a **structured language**.

Key Feature of Structured Languages: Compartmentalization

The most important feature of a structured language is **compartmentalization of code and data**.

Compartmentalization

Compartmentalization means dividing a program into **smaller independent sections**, where each section performs a specific task.

This approach offers several advantages:

- Improves program organization
- Simplifies debugging
- Enhances code reuse
- Reduces errors

Each section of the program can be developed and tested separately.

Role of Subroutines in Structured Programming

One way to achieve compartmentalization in C is by using **subroutines**, also called **functions**.

A **function** is a block of code designed to perform a specific task.

Functions allow programmers to divide large programs into smaller modules.

Advantages of Using Functions

- Programs become easier to understand
- Code can be reused multiple times
- Errors are easier to locate
- Large programs become manageable

Example:

```
#include <stdio.h>
```

```
void greet()
{
    printf("Hello World");
}
```

```
int main()
{
    greet();
    return 0;
}
```

In this example:

- main() is the main function.
- greet() is a separate function performing a specific task.

Use of Local Variables

Structured programming also encourages the use of **local variables**.

A **local variable** is a variable that is declared inside a function and can only be accessed within that function.

Example:

```
void calculate()
{
    int x = 10; // local variable
}
```

Advantages of local variables:

- Prevent interference with other parts of the program
- Reduce unexpected side effects
- Improve program reliability

Problems with Global Variables

A **global variable** is accessible throughout the entire program.

Example:

```
int x = 10;
```

Excessive use of global variables can cause several problems:

- Unexpected changes in variable values
- Difficult debugging
- Increased chances of program errors

Therefore, structured programming encourages **limited use of global variables**.

Control Structures in Structured Languages

Structured programming relies on **control structures** instead of unorganized jumps.

C provides several control structures:

1. Selection Structures

Used for decision making.

Examples:

- if
- if-else
- switch

Example:

```
if(x > 0)
{
    printf("Positive number");
}
```

2. Iteration Structures (Loops)

Used for repeating tasks.

Examples:

- for
- while
- do-while

Example:

```
for(int i = 0; i < 5; i++)  
{  
    printf("%d", i);  
}
```

3. Sequential Execution

Instructions are executed one after another.

Example:

```
printf("Start");  
printf("End");
```

Use of goto Statement

Older programming languages relied heavily on the **goto statement** for program control.

However, structured programming discourages the use of goto because it can lead to **spaghetti code** (a confusing and tangled program structure).

C allows the goto statement but recommends using structured control statements instead.

Examples of Structured and Non-Structured Languages

Programming languages can be categorized as structured or non-structured.

Non-Structured Languages

- FORTRAN
- BASIC
- COBOL

These languages often rely heavily on **goto statements**.

Structured Languages

- Pascal
- Ada
- C
- C++
- Java

Structured languages use **control structures and modular programming techniques**.

Importance of Structured Programming

Structured programming has several advantages:

1. Improves readability of programs
2. Simplifies debugging and testing
3. Makes programs easier to modify
4. Supports modular program design
5. Reduces program complexity

Because of these advantages, structured programming has become the **standard approach for modern software development**.

FUNCTIONS AS THE MAIN STRUCTURAL COMPONENT IN C

The main structural unit in C is the **function**.

All program activities occur inside functions.

Characteristics of C functions:

- Each function performs a specific task
- Functions operate independently
- Programs can contain multiple functions
- The main() function is the starting point of execution

Example:

```
int main()
{
    printf("Program started");
    return 0;
}
```

Functions help build large programs by dividing them into smaller logical units.

CODE BLOCKS IN C

Another way to structure programs in C is through **code blocks**.

A **code block** is a group of related statements enclosed within **curly braces {}**.

Example:

```
if(x < 10)
{
    printf("Too low, try again\n");
    scanf("%d", &x);
}
```

Here:

- The two statements inside {} form a **code block**.
- Both statements are executed together if the condition is true.

Benefits of code blocks:

- Organize program logic
- Improve readability
- Allow multiple statements to behave as one unit

Advantages of Code Blocks

Code blocks help programmers:

- Implement algorithms clearly
- Organize program statements logically
- Reduce program complexity
- Improve program efficiency

They also make it easier to understand the **logical flow of a program**.

C AS A PROGRAMMER'S LANGUAGE

Not all programming languages are designed primarily for programmers. Some languages are developed so that **non-programmers or beginners can easily understand and write programs**.

However, the **C programming language was designed specifically for professional programmers**. It provides the flexibility, efficiency, and control that programmers require to develop powerful and efficient software.

Because of this focus on the needs of programmers, C is often referred to as a **programmer's language**.

Languages Designed for Non-Programmers

Some programming languages were created mainly to make programming easier for people who were **not professional programmers**.

Two well-known examples are **COBOL** and **BASIC**.

COBOL (Common Business-Oriented Language)

COBOL was designed mainly for **business data processing** applications.

Characteristics of COBOL:

- Uses English-like statements
- Easy for non-programmers to read
- Designed for business professionals

Example (COBOL style):

```
ADD SALARY TO TOTAL-SALARY
```

The goal of COBOL was not to improve programming efficiency but to make programs **readable by managers and business analysts**.

BASIC (Beginner's All-Purpose Symbolic Instruction Code)

BASIC was designed to allow **beginners and non-programmers** to learn programming easily.

Characteristics of BASIC:

- Simple syntax
- Easy to learn
- Suitable for simple applications

Example:

```
PRINT "HELLO"
```

Although BASIC is simple and beginner-friendly, it lacks the power and flexibility needed for complex system programming.

C Designed for Programmers

Unlike COBOL and BASIC, **C was created by experienced programmers for professional programmers**.

C was developed at **Bell Laboratories** by **Dennis Ritchie**, who designed the language to meet the needs of **system programmers working on the UNIX operating system**.

C provides several features that programmers prefer:

- Few restrictions
- Efficient execution
- Flexible programming style
- Compact set of keywords
- Powerful low-level capabilities

Because of these features, C quickly became one of the most popular programming languages among professional programmers.

Features that Make C a Programmer's Language

Several characteristics make C especially suitable for programmers

1. Minimal Restrictions

C gives programmers significant freedom in how they write programs.

Unlike some high-level languages that enforce strict rules, C allows programmers to:

- Control memory directly
- Perform type conversions
- Manipulate pointers

This flexibility allows experienced programmers to create highly optimized programs.

2. Block Structure

C uses **structured programming techniques**, which help organize programs logically.

Programs can be divided into **blocks and functions**, making them easier to manage and understand.

Example:

```
if(x > 0)
{
    printf("Positive number");
}
```

Here, the statements inside { } form a **code block**.

3. Stand-Alone Functions

The main building block of a C program is the **function**.

Functions allow programmers to divide large programs into smaller modules.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Benefits of functions:

- Code reuse
- Better program organization
- Easier debugging

4. Compact Set of Keywords

C uses a **small number of keywords** compared to many other programming languages.

For example:

- C89 defines **32 keywords**
- C99 adds **5 additional keywords**

Examples of C keywords:

```
int
if
while
for
return
switch
```

This compact design makes the language **efficient and easy to implement**.

C vs Assembly Language

One of the major reasons for the success of C is that it can often **replace assembly language**.

Assembly Language

Assembly language is a **low-level programming language** that directly represents machine instructions.

Example (assembly style):

```
MOV AX, BX
```

```
ADD AX, 5
```

Advantages:

- Maximum control over hardware
- Highly efficient programs

Disadvantages:

- Difficult to write
- Difficult to debug
- Hard to maintain
- Not portable across different CPUs

Assembly programs often become **spaghetti code**, which means a confusing mixture of jumps and instructions.

Advantages of C Over Assembly

C offers several advantages compared to assembly language:

Feature	Assembly Language	C Language
Ease of Programming	Difficult	Easier
Structure	Poor	Structured
Portability	Very low	High
Development Speed	Slow	Faster

Thus, C provides **the efficiency of assembly language combined with the structure of high-level languages**.

Early Use of C

Initially, C was primarily used for **system programming**.

System programming involves writing software that controls or supports the operating system.

Examples include:

- Operating systems
- Compilers
- Linkers

- Editors
- Device drivers

One of the most famous examples is the **UNIX operating system**, which was rewritten in C.

Growth of C

As C became more popular, programmers began using it for many other types of applications.

Reasons for its popularity include:

- High efficiency
- Portability
- Flexibility
- Structured programming support

Today, C is used in a wide range of applications including:

- Operating systems
- Embedded systems
- Real-time systems
- Database systems
- System utilities

C and C++

When C++ was developed, some programmers believed that C would eventually disappear.

However, this did not happen.

There are several reasons why C is still widely used.

Reasons Why C Still Exists

1. Embedded Systems

Many embedded systems such as microcontrollers are still programmed using C because it provides direct hardware access.

Examples:

- Automotive control systems

- Medical devices
- Industrial machines

2. Existing Software

A large portion of the world's software infrastructure is written in C.

These programs still need to be **maintained and updated**, which keeps C relevant.

3. Continuous Improvements

The development of the **C99 standard** introduced new features and improvements, ensuring that C remains modern and useful.

Importance of C

C is considered one of the **most important programming languages in computer science**.

It has influenced many modern languages such as:

- C++
- Java
- Python
- C#

Because of its efficiency and flexibility, C **continues to** be widely used in professional programming.

COMPILERS VS INTERPRETERS

A **programming language defines how a program is written**, but it does not determine **how the program will be executed**.

Programs written in high-level languages must be translated into **machine language** before a computer can execute them.

There are **two primary methods** used to execute programs:

1. **Compilation**
2. **Interpretation**

Although any programming language can theoretically be compiled or interpreted, some languages are **better suited for one method than the other**.

For example:

- **C language** → **designed to be compiled**
- **Java** → **largely interpreted**
- **Early BASIC** → **interpreted**

C programs are almost always executed using a **compiler**.

Interpreter

An **interpreter** is a program that reads and executes the **source code line by line**.

Instead of translating the entire program at once, an interpreter processes each instruction individually and immediately performs the operation.

Working of an Interpreter

1. The interpreter reads one **line** of **source code**.
2. It translates that line into **machine instructions**.
3. The instruction is **executed immediately**.
4. The interpreter moves to the next line.

Example

Earlier versions of **BASIC** used interpretation.

Example BASIC code:

```
PRINT "HELLO"
```

The interpreter reads the instruction and **executes it immediately**.

Characteristics of Interpreters

- Execution occurs **line by line**
- No separate executable file is created
- Errors are detected during execution

Advantages of an Interpreter

1. Easier debugging
2. Immediate execution
3. Suitable for learning environments

Disadvantages of an Interpreter

1. Slower execution
2. Requires interpreter each time the program runs
3. Less efficient for large programs

Compiler

A **compiler** is a program that translates the **entire source code of a program into machine code** before execution.

The output produced by a compiler is called **object code** or **machine code**.

After compilation, the computer executes the **compiled program directly**.

Working of a Compiler

1. The compiler reads the entire source code.
2. It translates the program into **object code**.
3. The object code is stored as an **executable program**.
4. The executable program can run independently.

Object Code

Object code is also known as:

- **Machine code**
- **Binary code**

It consists of **0s and 1s** that the computer hardware can directly execute.

Example of a Compiled Language

C programs are compiled using a **C compiler** such as:

- GCC
- Turbo C
- Clang

Example C program:

```
#include <stdio.h>
```

```
int main()  
{  
    printf("Hello World");
```

```
    return 0;  
}
```

Steps:

1. Write program
2. Compile program
3. Generate executable file
4. Run program

Performance Difference

In general:

- **Compiled programs run faster**
- **Interpreted programs run slower**

Reason:

Compilation occurs **only once**, but interpretation occurs **every time the program runs**.

Thus interpretation introduces additional overhead during execution.

Comparison: Compiler vs Interpreter

Feature	Compiler	Interpreter
Translation	Entire program at once	Line by line
Speed	Faster	Slower
Error Detection	After compilation	During execution
Output	Creates executable file	No separate executable
Example Languages	C, C++, Go	Python, BASIC

THE FORM OF A C PROGRAM

A **C program consists of one or more functions**. Each function performs a specific task within the program.

The most important function in any C program is the **main() function**, which acts as the starting point of program execution.

When a C program runs, execution **always begins with the main() function.**

Keywords in C

A **keyword** is a reserved word that has a special meaning in the C programming language.

Keywords cannot be used as:

- variable names
- function names
- identifiers

Example:

```
int
char
if
else
while
for
return
```

Keywords Defined by C89

The **C89 standard defines 32 keywords.**

These keywords form the core commands of the C language.

C89 Keywords

```
auto
break
case
char
const
continue
default
do
double
else
enum
extern
float
for
goto
if
```

int
long
register
return
short
signed
sizeof
static
struct
switch
typedef
union
unsigned
void
volatile
while

Keywords Added by C99

The **C99 standard introduced additional keywords.**

Examples include:

_Bool
_Complex
_Imaginary
inline
restrict

These keywords provide additional functionality for modern C programming.

Extended Keywords

Some C compilers provide **non-standard keywords** that depend on the system or processor architecture.

Examples:

asm
cdecl
far
near
interrupt

pascal
huge

These keywords are often used in **system programming or embedded systems**.

Case Sensitivity in C

C is a **case-sensitive language**.

This means uppercase and lowercase letters are treated differently.

Example:

else → keyword
ELSE → not a keyword

Thus:

int age;
is different from
INT age;

The main() Function

Every C program must contain the **main() function**.

The main() function is where the program execution begins.

Example:

```
#include <stdio.h>
```

```
int main()  
{  
    printf("Hello World");  
    return 0;  
}
```

Explanation:

- #include <stdio.h> → includes standard input-output library
- main() → starting function of the program
- printf() → displays output
- return 0; → ends the program

GENERAL STRUCTURE OF A C PROGRAM

The general form of a C program is:

Global Declarations

```
int main(parameter list)
{
    statements
}
```

```
return_type function1(parameter list)
{
    statements
}
```

```
return_type function2(parameter list)
{
    statements
}
```

Explanation of Structure

Global Declarations

Variables or functions declared outside all functions.

Example:

```
int x;
```

main() Function

The main function is the **entry point of the program**.

All programs begin execution from this function.

User-Defined Functions

Programs can include additional functions.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

These functions help divide large programs into smaller manageable modules.

THE LIBRARY AND LINKING IN C

In the C programming language, it is technically possible to write a program using only the **keywords and basic syntax of the language**. However, such programs would be extremely limited because the C language itself does not include built-in keywords for performing many common operations such as:

- Input and output operations
- Mathematical calculations
- String and character manipulation

To perform these tasks, C uses **predefined functions stored in libraries**.

These libraries contain functions that perform commonly used operations, allowing programmers to write programs more efficiently.

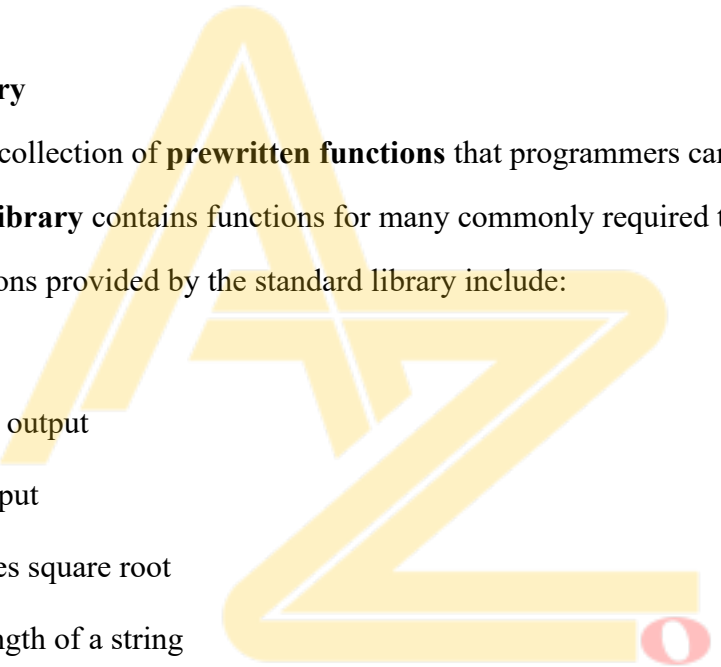
C Standard Library

A **library** in C is a collection of **prewritten functions** that programmers can use in their programs.

The **C Standard Library** contains functions for many commonly required tasks.

Examples of functions provided by the standard library include:

Function Purpose



printf()	Displays output
scanf()	Reads input
sqrt()	Calculates square root
strlen()	Finds length of a string
pow()	Calculates power of a number

Example program using library functions:

```
#include <stdio.h>
#include <math.h>

int main()
{
    printf("Square root: %f", sqrt(16));
    return 0;
}
```

Here:

- `stdio.h` provides input/output functions.
- `math.h` provides mathematical functions.

Importance of the Standard Library

The C standard library is important because it:

1. Reduces programming effort
2. Provides reliable and tested functions
3. Improves program development speed
4. Allows code reuse
5. Provides portability across different systems

Instead of writing complex functions from scratch, programmers can simply **use the existing library functions**.

Minimal Standard Library

The C language standard specifies a minimum set of library functions that must be supported by all C compilers.

However, most compilers provide **additional libraries and functions** beyond the standard set.

For example:

- The standard C library does not include graphics functions.
- Some compilers include additional libraries for graphics programming.

This means the exact library functions available may vary depending on the compiler.

Linking Process in C

When a C program uses a library function, the compiler does not directly include the function code in the program.

Instead, the compiler records the **name of the library function** used in the program.

Later, a program called the **linker** connects the program code with the required library functions.

This process is called **linking**.

Role of the Linker

The **linker** is responsible for combining different pieces of program code to produce the final executable program.

It performs the following tasks:

1. Combines the **object code generated by the compiler**
2. Connects the program with **library functions**
3. Generates the **final executable file**

Thus, linking ensures that all functions used in the program are properly connected.

Steps in the Linking Process

The linking process typically occurs after compilation and involves the following steps:

1. The programmer writes the source code.
2. The compiler converts the source code into **object code**.
3. The linker connects the object code with the **required library functions**.
4. The final **executable program** is created.

Relocatable Format

Functions stored in libraries are usually stored in a **relocatable format**.

This means that the **exact memory addresses of instructions are not fixed**.

Instead, only **offset information** is stored.

During the linking process:

- The linker assigns the actual memory addresses.
- The program and library functions are combined correctly.

This allows programs to be loaded into different memory locations without modification.

User-Defined Libraries

In addition to using standard libraries, programmers can also create **their own libraries**.

If a programmer writes functions that are used frequently, those functions can be stored in a library and reused in other programs.

Example:

A programmer may create a library for:

- Mathematical utilities

- File handling
- Graphics operations

This improves code reuse and reduces development time.

GENERAL FORM OF A C PROGRAM

A C program usually consists of several components, including global declarations, the main function, and user-defined functions.

General Structure

Global declarations

```
int main(parameter list)
{
    statement sequence
}

return_type f1(parameter list)
{
    statement sequence
}

return_type f2(parameter list)
{
    statement sequence
}
...
return_type fN(parameter list)
{
    statement sequence
}
```



Explanation of Program Structure

1. Global Declarations

These are variables or function declarations defined outside any function.

Example:

```
int x;
```

These variables can be accessed throughout the program.

2. main() Function

The **main() function** is the starting point of program execution.

Every C program must contain exactly one main() function.

Example:

```
int main()
{
    printf("Hello World");
    return 0;
}
```

3. User-Defined Functions

Programs can include multiple user-defined functions.

Each function performs a specific task and helps divide the program into smaller modules.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Advantages of Using Libraries

Using libraries in C offers several advantages:

1. Saves programming time
2. Reduces code complexity
3. Improves program reliability
4. Encourages code reuse
5. Simplifies program maintenance

Libraries act as **building blocks** for constructing complex programs.

SEPARATE COMPILE

In small C programs, the entire program is usually written in a **single source file**. However, as programs grow larger and more complex, maintaining everything in one file becomes difficult.

To solve this problem, C supports **separate compilation**, which allows a program to be divided into **multiple source files** that can be compiled independently.

Separate compilation is an important concept in **large-scale software development** because it improves program organization and reduces compilation time.

Concept of Separate Compilation

Separate compilation means dividing a large program into **multiple modules (source files)** and compiling each module independently.

Each module may contain:

- Functions
- Variables
- Program logic related to a specific task

After all files are compiled individually, they are **linked together to form the final executable program**.

Example Structure of Separate Compilation

A large program may be divided into several files such as:

main.c
math.c
input.c
output.c

Each file contains related functions.

Example:

main.c

```
#include <stdio.h>
```

```
int add(int, int);
```

```
int main()
{
    printf("%d", add(5,3));
    return 0;
}
```

math.c

```
int add(int a, int b)
{
    return a + b;
}
```

These files are compiled separately and then linked together.

Advantages of Separate Compilation

Separate compilation provides several important advantages.

1. Reduced Compilation Time

If one file is modified, only that file needs to be recompiled instead of recompiling the entire program. This significantly reduces compilation time for large projects.

2. Better Program Organization

Large programs can be divided into logical modules.

Each module performs a specific task, making the program easier to understand and maintain.

3. Team Collaboration

Separate compilation allows **multiple programmers to work on the same project simultaneously**.

Each programmer can work on different modules without interfering with others.

4. Code Reusability

Modules developed for one program can be reused in other programs.

Compiling a C Program

Creating an executable C program generally involves **three main steps**.

Step 1: Writing the Program

The programmer writes the source code using a **text editor**.

The program is saved as a file with the **.c extension**.

Example:

```
program.c
```

The source file contains the human-readable C code.

Step 2: Compiling the Program

The compiler translates the **source code into object code**.

Object code is machine-readable code but not yet executable.

Example compilation command:

```
gcc program.c
```

The compiler checks the program for syntax errors during this stage.

Step 3: Linking the Program

After compilation, the **linker combines the object code with required library functions**.

This process produces the final **executable program**.

For example:

program.exe

Integrated Development Environments (IDEs)

Most modern C compilers provide an **Integrated Development Environment (IDE)**.

An IDE usually includes:

- Text editor
- Compiler
- Linker
- Debugger

Examples of IDEs:

- Code::Blocks
- Dev-C++
- Visual Studio
- Eclipse

These environments simplify the program development process.

C's Memory Map

When a C program is compiled and executed, it uses several different **memory regions**.

A typical C program uses **four logical memory regions**.

1. Code section
2. Global variable section
3. Stack
4. Heap

Each region serves a specific purpose in program execution.

1. Code Section (Text Segment)

The **code section** stores the compiled program instructions.

This region contains the **executable machine code** that the CPU executes.

Characteristics:

- Usually read-only
- Contains compiled instructions
- Does not change during execution

2. Global Variable Section

This memory region stores **global variables and static variables**.

Global variables are accessible throughout the entire program.

Example:

```
int x = 10;
```

These variables remain in memory **throughout the program's execution**.

3. Stack

The **stack** is a region of memory used during program execution.

It stores:

- Function parameters
- Local variables
- Return addresses of function calls
- Temporary data

Example:

```
void function()  
{  
    int a = 5;  
}
```

The variable a is stored in the stack.

Characteristics of the stack:

- Automatically managed
- Last-In First-Out (LIFO) structure
- Memory allocated and released automatically

4. Heap

The **heap** is a region of memory used for **dynamic memory allocation**.

Memory in the heap is allocated manually using functions such as:

- malloc()
- calloc()
- realloc()
- free()

Example:

```
int *ptr;  
ptr = (int*) malloc(sizeof(int));
```

Characteristics of the heap:

- Dynamically allocated memory
- Managed by the programmer
- Used when memory size must be determined at runtime

Conceptual Memory Map of a C Program

The memory structure of a C program can be visualized as:

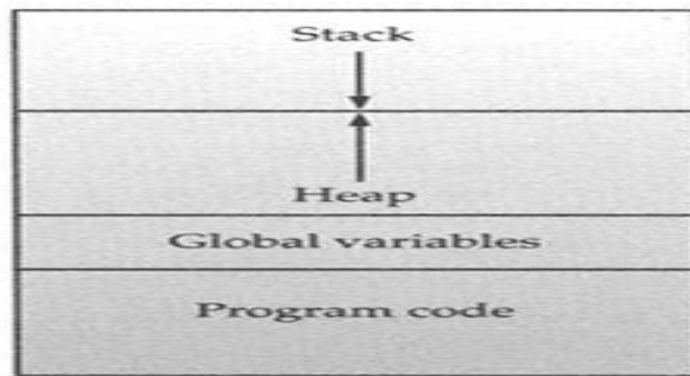


Figure 1-2
Conceptualized memory
map of a C program

The stack and heap grow toward each other as the program executes.

C VS C++

C++ is an extension of the C programming language.

It was developed to support **object-oriented programming concepts** while maintaining compatibility with C.

C++ was designed by **Bjarne Stroustrup** at Bell Laboratories.

Relationship Between C and C++

C++ was built on the foundation of C.

Therefore:

$C \subset C++$

This means **C is a subset of C++**, and most C programs can be compiled using a C++ compiler.

Differences Between C and C++

Feature	C	C++
Programming Style	Procedural	Object-Oriented
Concepts	Functions and procedures	Classes and objects
Data Hiding	Limited	Supported

Feature	C	C++
Inheritance	Not supported	Supported
Polymorphism	Not supported	Supported

File Extensions

By convention, file extensions are different for C and C++.

Language Extension

C .c

C++ .cpp

If a file has the .cpp extension, the compiler treats it as a **C++ program**.

Therefore, when compiling C programs, the correct extension .c must be used.

Importance of C Today

Even though C++ introduced object-oriented programming features, C continues to be widely used.

Reasons include:

- Efficient execution
- Direct hardware access
- Simplicity
- Large existing codebase

C is still widely used in:

- Operating systems
- Embedded systems
- System software
- Device drivers

EXPRESSIONS IN C

An **expression** is one of the most fundamental elements of the C programming language.

Expressions are used to perform **computations, comparisons, and logical operations** in a program.

In C, expressions are more **flexible and powerful** compared to many other programming languages.

An expression is formed using **two main components**:

1. **Data**
2. **Operators**

Data in Expressions

Data may be represented in several forms:

- **Variables**
- **Constants**
- **Values returned by functions**

Example:

`x = a + b;`

Here:

- `a` and `b` → data (variables)
- `+` → operator
- `a + b` → expression

Basic Data Types in C

The **C89 standard** defines **five basic data types**, which form the foundation for all other types.

Data Type	Keyword	Description
Character	<code>char</code>	Stores a single character
Integer	<code>int</code>	Stores whole numbers
Floating point	<code>float</code>	Stores decimal numbers
Double floating point	<code>double</code>	Stores large decimal numbers
Valueless	<code>void</code>	Represents no value

These basic types are used to create more complex data types.

Size of Data Types

The size and range of data types depend on the **compiler and processor architecture**.

However, some properties are standard:

- char is always **1 byte**
- int size depends on the environment

Example:

Environment Size of int

16-bit system 16 bits

32-bit system 32 bits

Because sizes vary across systems, C specifies **minimum ranges rather than exact sizes**.

Additional Data Types in C99

The C99 standard introduced **three additional data types**:

Data Type Purpose

_Bool Boolean values (true/false)

_Complex Complex numbers

_Imaginary Imaginary numbers

These types extend C's capabilities for **mathematical and scientific computing**.

Floating Point Numbers

Floating point numbers represent **decimal or real numbers**.

The minimum range specified by the C standard is approximately:

1E-37 to 1E+37

Precision of Floating Types

Type	Minimum Precision
------	-------------------

float	6 digits
-------	----------

double	10 digits
--------	-----------

long double	10 or more digits
-------------	-------------------

Floating point representation may vary depending on the system.

The void Data Type

The void type represents **no value**.

It is used in two main situations:

1. Functions Returning No Value

Example:

```
void display()
{
    printf("Hello");
}
```

2. Generic Pointers

Example:

```
void *ptr;
```

This pointer can store the address of any data type.

Modifying Basic Data Types

C allows **type modifiers** to alter the meaning or range of basic data types.

The four modifiers are:

signed
unsigned
long
short

These modifiers adjust the **range and storage size** of variables.

Valid Type Modifications

For int

int can be modified using:

- signed
- unsigned
- short
- long

Examples:

```
signed int a;  
unsigned int b;  
short int c;  
long int d;
```

For char

char can be modified using:

- signed
- unsigned

Example:

```
signed char ch1;  
unsigned char ch2;
```

For double

double can be modified using:

long double

Example:

```
long double num;
```

Default Behavior of signed

Using signed with integers is usually unnecessary because **integers are signed by default**.

Example:

```
signed int x;  
int x;
```

Both declarations mean the same thing.

However, signed is sometimes used with char because some systems treat char as **unsigned by default**.

Signed vs Unsigned Integers

The difference between signed and unsigned integers lies in how the **most significant bit (MSB)** is interpreted.

Signed Integers

- Highest bit represents the **sign**
- 0 → positive number
- 1 → negative number

Example range (16-bit):

-32768 to +32767

Unsigned Integers

- All bits represent the magnitude
- Only **positive values**

Example range (16-bit):

0 to 65535

Thus, unsigned integers can store **larger positive values**.

Two's Complement Representation

Negative numbers in C are typically represented using **two's complement representation**.

Steps:

1. Reverse all bits of the number
2. Add 1
3. Set the sign bit

This method allows computers to perform arithmetic efficiently.

Type Modifiers Without Base Type

When a modifier appears alone, **int** is assumed automatically.

Example equivalence:

Specifier Equivalent

signed signed int

unsigned unsigned int

long long int

Specifier Equivalent

short short int

Although the compiler assumes int, it is good programming practice to **write the full type explicitly**.

Identifier Names in C

An **identifier** is the name given to:

- Variables
- Functions
- Arrays
- Labels
- User-defined items

Identifiers help programmers refer to different program elements.

Example:

```
int count;  
float total_balance;
```

Here:

- count
- total_balance

are identifiers.

Rules for Naming Identifiers

C imposes several rules for identifier names.

1. The first character must be a **letter or underscore**.
2. Remaining characters may be:
 - letters
 - digits
 - underscores
3. Spaces are not allowed.
4. Special symbols are not allowed.

Valid Identifier Examples

count
test23
high_balance
_value

Invalid Identifier Examples

1count (starts with digit)
hi!there (contains special character)
high...balance (contains invalid characters)

Identifier Length

C allows identifiers of **any length**, but not all characters may be significant.

Two types of identifiers exist:

1. External Identifiers

Used for **functions and global variables shared between files**.

Example:

int total;

In C89:

- First **6 characters** are guaranteed significant.

In C99:

- First **31 characters** are significant.

2. Internal Identifiers

Used for **local variables within a function**.

Example:

int counter;

In C89:

- First **31 characters** are significant.

In C99:

- First **63 characters** are significant.

Case Sensitivity in Identifiers

C is a **case-sensitive language**.

This means uppercase and lowercase letters are treated differently.

Example:

```
count  
Count  
COUNT
```

All three are **different identifiers**.

Restrictions on Identifiers

An identifier:

- **Cannot be a C keyword**

Example:

```
int, while, return
```

These are reserved words.

Also, identifiers **should not use names of standard library functions** to avoid confusion.

VARIABLES IN C

A **variable** in C is a named location in memory that is used to store a value which can be modified during the execution of a program. Each variable has a specific **data type**, which determines the type of value it can hold.

Variables act as containers that store data such as numbers, characters, or floating-point values.

Syntax of Variable Declaration

A variable must be declared before it is used in a program.

General syntax:

```
data_type variable_name;
```

Example:

```
int count;  
float balance;  
char grade;
```

Here:

- int, float, char → data types
- count, balance, grade → variable names

Multiple Variable Declaration

More than one variable can be declared in a single statement.

Example:

```
int a, b, c;  
float x, y;
```

Initialization of Variables

A variable may also be assigned a value during declaration.

Example:

```
int x = 10;  
float price = 25.5;
```

Characteristics of Variables

1. Each variable occupies memory.
2. The value stored in a variable can change during program execution.
3. Variables must follow identifier naming rules.
4. Variables must be declared before use.

Example Program

```
#include<stdio.h>
```

```
int main()  
{  
    int a = 10;  
    int b = 20;  
    int sum;
```

```
sum = a + b;

printf("Sum = %d", sum);
return 0;
}
```

WHERE VARIABLES ARE DECLARED

Variables in C can be declared in different locations depending on how they are used in the program.

There are three main places where variables can be declared:

1. Inside functions
2. As parameters of functions
3. Outside all functions

1. Inside Functions

Variables declared inside functions are called **local variables**.

Example:

```
void func()
{
    int x;
}
```

These variables can only be accessed inside that function.

2. Function Parameters

Variables declared in the function parameter list receive values from function calls.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Here a and b are parameters.

3. Outside Functions

Variables declared outside all functions are called **global variables**.

Example:

```
int count;
```

These variables can be accessed by all functions in the program.

LOCAL VARIABLES

A **local variable** is a variable declared inside a function or block of code. It can only be accessed within the block in which it is declared.

Characteristics

1. Declared inside functions.
2. Accessible only within that function.
3. Created when the block begins.
4. Destroyed when the block ends.
5. Stored in stack memory.

Example

```
#include<stdio.h>
```

```
void func()
{
    int x = 10;
    printf("%d", x);
}
```

```
int main()
{
    func();
    return 0;
}
```

Here x is a local variable of func().

Local Variables in Different Functions

```
void func1()
{
    int x = 10;
}
```

```
void func2()
{
    int x = 20;
}
```

Both functions have separate variables even though the name is the same.

BLOCK SCOPE

A **block** is a group of statements enclosed within curly braces { }. Variables declared inside a block can only be used inside that block.

Example of a Block

```
{
    int x = 5;
}
```

The variable x is visible only within this block.

Example Program

```
#include<stdio.h>

int main()
{
    int x = 100;

    if(x == 100)
    {
        int y = 200;
        printf("%d %d", x, y);
    }
    printf("%d", x);
    return 0;
}
```

Explanation:

- x → accessible everywhere in main()
- y → accessible only inside the if block

Advantages of Block Scope

1. Prevents variable conflicts.
2. Improves program structure.
3. Enhances modular programming.

FORMAL PARAMETERS

Formal parameters are variables declared in the **parameter list of a function**. They receive values passed to the function when it is called.

Syntax

```
return_type function_name(type parameter1, type parameter2)
```

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Here a and b are formal parameters.

Working of Formal Parameters

When the function is called:

```
add(5, 3);
```

- a receives 5
- b receives 3

Example Program

```
#include<stdio.h>
int multiply(int a, int b)
{
    return a * b;
}
int main()
{
    int result;
```

```
result = multiply(4,5);  
printf("%d", result);  
return 0;  
}
```

Characteristics

1. Declared in function definition.
2. Act as local variables.
3. Exist only during function execution.

GLOBAL VARIABLES

A **global variable** is a variable declared outside all functions. It can be accessed by any function in the program.

Example

```
#include<stdio.h>  
int count = 10;  
void display()  
{  
    printf("%d", count);  
}  
  
int main()  
{  
    display();  
    return 0;  
}
```

Here count is a global variable.

Characteristics

1. Declared outside all functions.
2. Accessible throughout the program.
3. Stored in global memory area.
4. Exists for the entire program execution.

Advantages

- Useful for sharing data among functions.

Disadvantages

1. Hard to debug.
2. Can cause unexpected side effects.
3. Reduces modularity.

SCOPE OF VARIABLES

The **scope of a variable** refers to the region of a program where the variable can be accessed. C provides four types of scope.

1. File Scope

Variables declared outside functions have **file scope**.

Example:

```
int total;
```

Accessible from the declaration point to the end of the file.

2. Block Scope

Variables declared inside { }.

Example:

```
{  
    int x;  
}
```

Visible only within the block.

3. Function Prototype Scope

Variables declared in function prototypes.

Example:

```
int func(int a, int b);
```

Scope exists only in the prototype declaration.

4. Function Scope

Labels used with goto statements.

Example:

```

void test()
{
start:
    printf("Hello");
}

```

Label start is visible throughout the function.

Scope	Meaning
File scope	Starts at the beginning of the file (also called a <i>translation unit</i>) and ends with the end of the file. It refers only to those identifiers that are declared outside of all functions. File scope identifiers are visible throughout the entire file. Variables that have file scope are global.

Scope	Meaning
Block scope	Begins with the opening { of a block and ends with its associated closing }. However, block scope also extends to function parameters in a function definition. That is, function parameters are included in a function's block scope. Variables with block scope are local to their block.
Function prototype scope	Identifiers declared in a function prototype; visible within the prototype.
Function scope	Begins with the opening { of a function and ends with its closing }. Function scope applies only to labels. A label is used as the target of a goto statement, and that label must be within the same function as the goto .

TYPE QUALIFIERS IN C

In the C programming language, **type qualifiers** are used to control how variables can be **accessed or modified** in a program. They provide additional information about the behavior of variables.

Type qualifiers are placed **before the data type** in a declaration.

C89 defines two type qualifiers:

1. **const**
2. **volatile**

Later, the **C99 standard introduced another qualifier called restrict**, but the main qualifiers used in most programs are const and volatile.

General syntax:

qualifier data_type variable_name;

Example:

```
const int a;  
volatile int counter
```

1. const Type Qualifier

Definition

The **const qualifier** specifies that the value of a variable **cannot be modified by the program after initialization**.

A variable declared as const must either:

- be initialized when declared, or
- receive its value from some **external or hardware source**.

The compiler may place const variables in **read-only memory (ROM)**.

Syntax

```
const data_type variable_name = value;
```

Example:

```
const int a = 10;
```

Here:

- a is a constant integer variable.
- Its value **cannot be changed during program execution**.

Example of incorrect modification:

```
const int a = 10;  
a = 20; // ERROR
```

This will generate a **compile-time error**.

Example Program

```
#include<stdio.h>
```

```
int main()  
{  
    const int x = 50;  
  
    printf("%d", x);  
  
    return 0;  
}
```

Output:

50

If we try to change x, the compiler will produce an error.

const with Pointers

The const qualifier is often used with **pointers in function parameters** to ensure that the function **does not modify the data being passed**.

Example:

```
void sp_to_dash(const char *str);
```

This means the function cannot modify the string pointed to by str.

Example Program Using const Pointer

```
#include<stdio.h>
```

```
void sp_to_dash(const char *str);
```

```
int main()
{
    sp_to_dash("this is a test");
    return 0;
}
```

```
void sp_to_dash(const char *str)
{
    while(*str)
    {
        if(*str == ' ')
            printf("-");
        else
            printf("%c", *str);

        str++;
    }
}
```

Output:

this-is-a-test

Here:

- The function prints dashes instead of spaces.
- The original string is **not modified** because str is declared as const.

Why const is Important

The const qualifier is useful because it:

1. Prevents accidental modification of variables.
2. Protects data from unintended changes.
3. Improves program safety.
4. Helps the compiler optimize memory usage.

Many **standard library functions** use const.

Example:

```
size_t strlen(const char *str);
```

This ensures that the strlen() function **cannot modify the string** passed to it.

2. volatile Type Qualifier

Definition

The **volatile qualifier** tells the compiler that a variable's value **may change at any time due to external factors**, not just by program instructions.

Therefore, the compiler **must not optimize access to this variable**.

Syntax

```
volatile data_type variable_name;
```

Example:

```
volatile int timer;
```

Why volatile is Needed

Normally, compilers optimize programs by assuming that variable values **do not change unless explicitly modified** in the program.

However, some variables may change due to:

- hardware devices

- interrupts
- operating system routines
- shared memory

The volatile qualifier prevents the compiler from making such assumptions.

Example Situation

Suppose a variable stores the **system clock time**, which is updated by the operating system.

Example:

```
volatile int system_time;
```

Even if the program does not change system_time, the compiler must **always read its latest value from memory**.

Characteristics of volatile Variables

1. Value may change unexpectedly.
2. Compiler cannot optimize the variable.
3. Memory is accessed **every time** the variable is used.
4. Commonly used in **embedded systems and hardware programming**.

Using const and volatile Together

A variable can be declared using **both qualifiers together**.

Example:

```
const volatile char *port = (const volatile char *)0x30;
```

Explanation:

- volatile → value may change due to hardware
- const → program itself cannot modify the value

This is common when accessing **hardware ports or device registers**.

STORAGE CLASS SPECIFIERS IN C

In C programming, variables have certain properties such as:

- **Scope** – where the variable can be accessed
- **Lifetime** – how long the variable exists in memory
- **Storage location** – where the variable is stored

- **Visibility** – which parts of the program can use the variable

These properties are controlled using **Storage Class Specifiers**.

A **storage class specifier** tells the compiler:

- where to store the variable
- how long the variable will exist
- which parts of the program can access it

A **storage class specifier** is a keyword used in a variable declaration that defines the **lifetime, scope, and storage location** of the variable.

Types of Storage Classes

C supports **four storage class specifiers**:

1. auto
2. register
3. static
4. extern

General Syntax

```
storage_class data_type variable_name;
```

Example:

```
static int count;  
extern int total;  
register int i;
```

Here:

- static, extern, register → storage class specifiers
- int → data type
- count, total, i → variable names

2. AUTO STORAGE CLASS

The **auto storage class** is the default storage class for **local variables declared inside functions**.

When a variable is declared inside a function without specifying any storage class, it automatically becomes an **auto variable**.

Syntax

```
auto int variable_name;
```

Example:

```
auto int x;
```

Characteristics of auto Variables

Property	Description
Scope	Local to the function
Lifetime	Exists only while function executes
Storage	Stack memory
Default value	Garbage value

Working of auto Variables

When a function starts executing:

- memory is allocated for auto variables

When the function ends:

- memory is destroyed automatically

Example Program

```
#include<stdio.h>
```

```
void display()
{
    auto int x = 10;
    printf("Value of x = %d\n", x);
}
```

```
int main()
{
    display();
    return 0;
}
```

Output

Value of x = 10

Example Showing Scope

```
#include<stdio.h>
```

```
void func()
{
    auto int a = 5;
    printf("%d\n", a);
}
```

```
int main()
{
    func();
    printf("%d\n", a); // error
}
```

Explanation:

- a exists only inside func()
- it cannot be accessed in main()

3. REGISTER STORAGE CLASS

Definition

The **register storage class** requests the compiler to store the variable in a **CPU register instead of main memory**.

Registers are **faster than memory**, so register variables provide **faster access**.

Syntax

```
register data_type variable_name;
```

Example:

```
register int i;
```

Characteristics

Property	Description
Storage location	CPU registers
Scope	Local
Lifetime	Function execution
Default value	Garbage

Property	Description
----------	-------------

Address operator	Not allowed
------------------	-------------

Why Register Variables Are Used

Register variables are used when:

- variable is used frequently
- loops require fast execution
- repeated calculations occur

Example Program

```
#include<stdio.h>
```

```
int power(register int m, register int e)
{
    register int temp = 1;

    for(; e > 0; e--)
        temp = temp * m;

    return temp;
}
```

```
int main()
{
    printf("%d", power(2,5));
    return 0;
}
```

Output

32

Explanation:

The variables m, e, and temp are stored in **CPU registers** for faster processing.

Important Rules

1. Register variables must be **local variables**.
2. Global register variables are **not allowed**.
3. The **address operator (&)** cannot be used with register variables.

Example:

```
register int x;  
printf("%p", &x); // ERROR
```

Reason: registers do not have memory addresses.

4. STATIC STORAGE CLASS

The static keyword has **two different behaviors**:

1. Static Local Variables
2. Static Global Variables

Static Local Variables

A **static local variable** is a local variable that **retains its value between function calls**.

Unlike normal local variables, static variables are **not destroyed after function execution**.

Syntax

```
static data_type variable_name;
```

Example:

```
static int count;
```

Characteristics

Property	Description
Scope	Local to function
Lifetime	Entire program
Initialization	Only once
Default value	0

Example Program

```
#include<stdio.h>
```

```
void counter()  
{  
    static int count = 0;  
    count++;  
}
```

```
    printf("Count = %d\n", count);  
}
```

```
int main()  
{  
    counter();  
    counter();  
    counter();  
}
```

Output

```
Count = 1  
Count = 2  
Count = 3
```

Explanation:

The variable count retains its value between function calls.

Static Global Variables

A **static global variable** is a global variable that is **accessible only within the file in which it is declared**.

Syntax

```
static int variable_name;
```

Example:

```
static int total;
```

Characteristics

Property	Description
Scope	Only within file
Lifetime	Entire program
Visibility	Hidden from other files

Example

File1.c

```
static int x = 10;
```

```
void display()
{
    printf("%d", x);
}
```

File2.c

```
extern int x; // ERROR
```

Explanation:

x cannot be accessed from another file because it is **static**.

EXTERN STORAGE CLASS

The extern keyword is used to **declare a variable that is defined in another file or location in the program**.

It tells the compiler:

“The variable exists somewhere else; just use it here.”

Syntax

```
extern data_type variable_name;
```

Example:

```
extern int total;
```

Declaration vs Definition

Concept	Meaning
---------	---------

Declaration	Tells compiler about variable
-------------	-------------------------------

Definition	Allocates memory
------------	------------------

Example:

```
extern int x; // declaration
int x = 10;   //definition
```

Example Program

```
#include<stdio.h>
```

```
int main()
{
    extern int first, last;

    printf("%d %d", first, last);
}
```

```
int first = 10;
int last = 20;
```

Output

10 20

Use in Multiple Files

File1.c

```
int x = 10;
```

File2.c

```
extern int x;
```

```
printf("%d", x);
```

Explanation:

- variable defined in File1
- accessed in File2 using extern

Comparison of Storage Classes

Storage Class	Scope	Lifetime	Storage
auto	Local	Function execution	Stack
register	Local	Function execution	CPU register
static	Local/File	Entire program	Static memory
extern	Global	Entire program	External

. CONSTANTS IN C

A **constant** is a value that **cannot be changed during the execution of a program.**

Once a constant value is defined, the program **cannot modify it**.

Constants are also called **literals**.

Example:

10

'A'

3.14

"Hello"

These values remain fixed during program execution

Types of Constants in C

C supports several types of constants:

1. Integer constants
2. Floating-point constants
3. Character constants
4. String constants
5. Escape sequence constants
6. Octal constants
7. Hexadecimal constants

1. Integer Constants

Definition

Integer constants are **whole numbers without a decimal point**.

They may be:

- positive
- negative
- zero

Examples:

10

0

-25

3500

Rules for Integer Constants

1. Must contain **only digits**
2. No decimal point allowed
3. May have + **or** – **sign**
4. Must not contain spaces

Example Program

```
#include<stdio.h>
```

```
int main()
{
    int a = 10;
    int b = -20;

    printf("%d %d", a, b);

    return 0;
}
```

Output

10 -20

Integer Suffixes

Sometimes we specify the **type of integer constant** using suffixes.

Suffix Meaning

U Unsigned

L Long

LL Long long

Examples

100U

35000L

999999LL

2. Floating-Point Constants

Definition

Floating-point constants represent **real numbers containing decimal points**.

Examples

3.14
0.5
-12.45

Scientific Notation

Floating numbers can also be written in **exponential form**.

Example

3.2e3

Meaning

$3.2 \times 10^3 = 3200$

Another example

$4.5e-2 = 0.045$

Floating-Point Suffixes

Suffix Meaning

F float

L long double

Examples

3.14F

12.5L

3. Character Constants

Definition

A **character constant** is a single character enclosed in **single quotes**.

Examples

'A'

'b'

'9'

'%'

Example Program

```
#include<stdio.h>
```

```
int main()  
{  
    char ch = 'A'
```

```
printf("%c", ch);

return 0;
}
```

Output

A

ASCII Representation

Each character corresponds to an ASCII value.

Example

'A' = 65

'B' = 66

'a' = 97

Program example

```
#include<stdio.h>
```

```
int main()
{
    char ch = 'A';

    printf("%d", ch);

    return 0;
}
```

Output

65

4. String Constants

Definition

A **string constant** is a sequence of characters enclosed in **double quotes**.

Examples

"Hello"

"Programming"

"This is C language"

Example Program



```
#include<stdio.h>
```

```
int main()
{
    printf("Welcome to C Programming");

    return 0;
}
```

Output

Welcome to C Programming

Character vs String

Character	String
'A'	"A"
Single quotes	Double quotes
Single character	Multiple characters

5. Escape Sequence Constants

Definition

Escape sequences represent **special characters** that cannot be typed directly.

They start with **backslash (\)**.

Common Escape Sequences

Escape Code Meaning

\n	New line
\t	Tab
\b	Backspace
\r	Carriage return
\f	Form feed
\	Backslash
'	Single quote

Escape Code Meaning

"	Double quote
\a	Alert sound

Example Program

```
#include<stdio.h>
```

```
int main()
{
    printf("Hello\nWorld");
}
```

Output

Hello
World

6. Octal Constants

Definition

Octal constants are **numbers expressed in base 8**.

They contain digits from **0 to 7**.

They start with **0**.

Examples

012
077
010

Example

012 = 10 (decimal)

7. Hexadecimal Constants

Definition

Hexadecimal numbers are **base 16 numbers**.

They use digits:

0-9
A-F

They start with:

0x

Examples

0x10

0xA1

0xFF

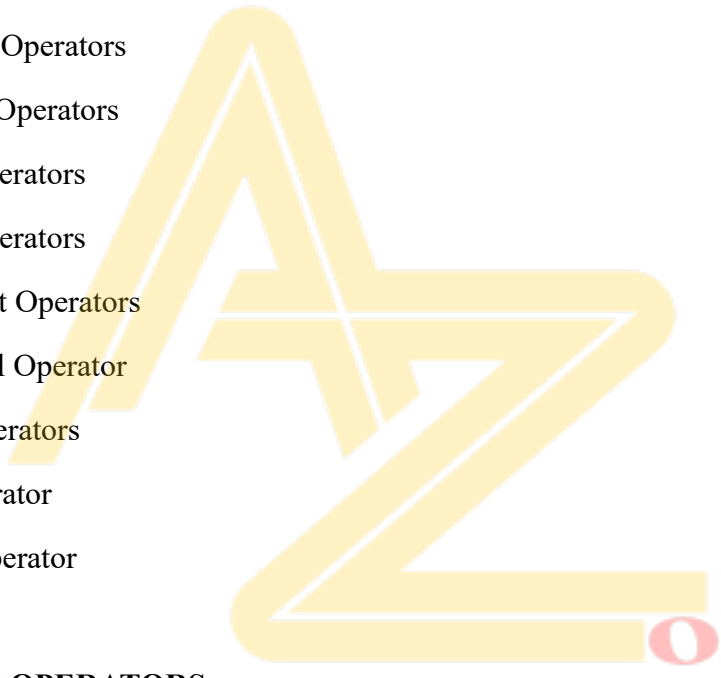
Example

0x10 = 16 (decimal)

TYPES OF OPERATORS IN C

Operators are symbols that perform operations on variables and constants.

C provides many operators. The important ones in this chapter are:

1. Arithmetic Operators
 2. Relational Operators
 3. Logical Operators
 4. Bitwise Operators
 5. Assignment Operators
 6. Conditional Operator
 7. Pointer Operators
 8. sizeof Operator
 9. Comma Operator
- 

1. ARITHMETIC OPERATORS

Arithmetic operators perform **mathematical calculations** such as addition, subtraction, multiplication, and division.

List of Arithmetic Operators

Operator Meaning

+	Addition
-	Subtraction
*	Multiplication

Operator Meaning

/	Division
%	Modulus (remainder)
++	Increment
--	Decrement

1. Addition Operator (+)

Definition

The addition operator adds two operands.

Syntax

result = operand1 + operand2

Example

```
#include<stdio.h>
int main()
{
    int a = 10;
    int b = 20;

    int sum = a + b;

    printf("Sum = %d", sum);

    return 0;
}
```

Output

Sum = 30

2. Subtraction Operator (-)

The subtraction operator subtracts one value from another.

Syntax

result = operand1 - operand2

Example

```
#include<stdio.h>
int main()
{
    int a = 50;
    int b = 20;

    int result = a - b;

    printf("Result = %d", result);

    return 0;
}
```

Output

Result = 30

Unary Minus

Unary minus changes the sign of a number.

Example

x = -10

Program

```
int a = 10;
int b = -a;
```

Result

b = -10

3. Multiplication Operator (*)

The multiplication operator multiplies two numbers.

Syntax

```
result = operand1 * operand2
```

Example

```
#include<stdio.h>
```

```
int main()
{
    int a = 6;
    int b = 5;

    printf("Multiplication = %d", a*b);

    return 0;
}
```

Output

Multiplication = 30

4. Division Operator (/)

Division divides one value by another.

Syntax

result = operand1 / operand2

Example

```
#include<stdio.h>
```

```
int main()
{
    int a = 10;
    int b = 2;

    printf("Division = %d", a/b);

    return 0;
}
```

Output

Division = 5

Integer Division

When both operands are integers, the result is an integer.

Example

$5 / 2 = 2$

The decimal part is removed.

5. Modulus Operator (%)

The modulus operator returns the **remainder of a division**.

Syntax

result = operand1 % operand2

Example

```
#include<stdio.h>
```

```
int main()
{
    int a = 10;
    int b = 3;

    printf("Remainder = %d", a % b);

    return 0;
}
```

Output

Remainder = 1

6. Increment Operator (++)

The increment operator increases the value of a variable by **1**.

Syntax

++variable

variable++

Example

```
#include<stdio.h>
```

```
int main()
{
    int x = 5;
    x++;
}
```

```
printf("%d", x);
```

```
    return 0;  
}
```

Output

6

Prefix Increment

`++x`

Increment happens **before use**.

Example

```
x = 10;  
y = ++x;
```

Result

```
x = 11  
y = 11
```

Postfix Increment

`x++`

Increment happens **after use**.

Example

```
x = 10;  
y = x++;
```

Result

```
x = 11  
y = 10
```

7. Decrement Operator (--)

Definition

The decrement operator decreases the value of a variable by **1**.

Syntax

--X

X--

Example

```
#include<stdio.h>
```

```
int main()
{
    int x = 10;

    x--;

    printf("%d", x);

    return 0;
}
```

Output

9

2. RELATIONAL OPERATORS

Definition

Relational operators compare two values.

They return:

1 → true

0 → false

List of Relational Operators

Operator Meaning

>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
==	Equal to
!=	Not equal

Example

```
#include<stdio.h>
```

```
int main()
{
    int a = 10;
    int b = 20;

    printf("%d\n", a>b);
    printf("%d\n", a<b);

    return 0;
}
```

Output

```
0
1
```

3. LOGICAL OPERATORS

Logical operators combine multiple conditions.

Types

Operator Meaning

&& Logical AND

! Logical NOT

Logical AND (&&)

True only if **both conditions are true**.

Example

(a>0 && a<10)

Program

```
#include<stdio.h>
int main()
{
    int a = 5;
```

```
if(a>0 && a<10)
    printf("Valid");

return 0;
}
```

Logical OR (||)

True if **at least one condition is true**.

Example

```
(a>10 || a<5)
```

Logical NOT (!)

Reverses the condition.

Example

```
!(a>10)
```

Truth Table

A	B	A && B	A B
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

4. BITWISE OPERATORS

Bitwise operators perform operations on **binary bits**.

List

Operator Meaning

&	Bitwise AND
	Bitwise OR
^	XOR
~	Complement
<<	Left shift

Operator Meaning

>> Right shift

Bitwise AND (&)

Example

5 = 0101

3 = 0011

Result

0001

Decimal = 1

Bitwise OR (|)

Example

5 = 0101

3 = 0011

Result

0111

Decimal = 7

XOR (^)

True only when bits are different.

Example

0101

0011

0110

Decimal = 6

Shift Operators

Left Shift (<<)

Moves bits left.

Example



5 << 1

Binary

0101 → 1010

Result

10

Right Shift (>>)

Moves bits right.

Example

8 >> 1

Result

4

5. ASSIGNMENT OPERATORS

Assignment operators assign values to variables.

Basic Assignment

variable = value

Example

x = 10

Compound Assignment

Operator Meaning

+= Add and assign

-= Subtract and assign

*= Multiply and assign

/= Divide and assign

%= Modulus and assign

Example

```
x += 5;
```

Equivalent

```
x = x + 5
```

6. CONDITIONAL OPERATOR (?:)

Definition

The conditional operator replaces simple **if-else statements**.

Syntax

```
condition ? expression1 : expression2
```

Example

```
#include<stdio.h>
```

```
int main()
{
    int x = 10;

    int y = x>5 ? 100 : 200;

    printf("%d", y);
}
```

Output

100

7. POINTER OPERATORS

Two important pointer operators exist.

&

*

Address Operator (&)

Returns the **memory address of a variable**.

Example

```
int x = 10;
int *p;
p = &x;
```

Dereference Operator (*)

Accesses the value stored at the address.

Example

*p

Program

```
#include<stdio.h>
```

```
int main()
{
    int x = 10;
    int *p = &x;

    printf("%d", *p);
}
```

Output

10

8. sizeof OPERATOR

sizeof returns the **size of a variable or data type in bytes.**

Syntax

```
sizeof(variable)
sizeof(type)
```

Example

```
#include<stdio.h>
```

```
int main()
{
    printf("%d\n", sizeof(int));
    printf("%d\n", sizeof(float));

    return 0;
}
```

Output

4
4

9. COMMA OPERATOR

Definition

The comma operator allows **multiple expressions to be evaluated in a single statement.**

Syntax

expression1 , expression2

The value of the **last expression becomes the result.**

Example

```
#include<stdio.h>
```

```
int main()
{
    int x;

    x = (3,5);

    printf("%d", x);

    return 0;
}
```

Output

5

Explanation

3 is evaluated

5 is returned



EXPRESSIONS IN C

An **expression in C** is a combination of **constants, variables, operators, and function calls** that produces a value.

Expressions are the **most fundamental building blocks of a C program.** Almost every statement in C involves an expression.

General Form

operand operator operand

or

variable = expression

Examples

$x + y$

$a * b + c$

$x = a + b$

$y = (a + b) * c$

Each of these expressions produces a value after evaluation.

Components of an Expression

An expression consists of the following elements:

1. Variables

Variables are memory locations that store data values.

Example

`int x = 10;`

Example expression

$x + 5$

2. Constants

Constants are fixed values that do not change during program execution.

Example

10

3.14

'A'

Example expression

$x + 10$

3. Operators

Operators perform operations on operands.

Examples

+

-

*
/
%

Example expression

$a + b * c$

4. Function Calls

Functions may also appear inside expressions.

Example

`result = sqrt(25);`

3. Types of Expressions

Expressions in C can be classified into several types.

1. Arithmetic Expressions

Arithmetic expressions perform mathematical operations.

Example

$a + b$
 $a * b + c$
 $(a + b) * c$

Example Program

```
#include<stdio.h>
int main()
{
    int a = 10;
    int b = 5;
    int result;
    result = a + b * 2;
    printf("Result = %d", result);

    return 0;
}
```

Explanation

$b * 2 = 10$
 $a + 10 = 20$

Output

Result = 20

2. Relational Expressions

Relational expressions compare two values.

They return:

1 → true

0 → false

Operators Used

>

<

>=

<=

==

!=

Example

a > b

x <= y

Example Program

```
#include<stdio.h>
int main()
{
    int a = 10;
    int b = 20;
    printf("%d", a < b);
    return 0;
}
```

Output

1

3. Logical Expressions

Logical expressions combine multiple conditions.

Logical Operators

&& AND

|| OR

! NOT

Example

$(a > 5) \&\& (a < 10)$

Example Program

```
#include<stdio.h>
int main()
{
    int a = 7;
    if(a > 5 && a < 10)
        printf("True");

    return 0;
}
```

Output

True

4. Order of Evaluation in Expressions

Expressions may contain several operators.

C follows **operator precedence rules** to determine the order of execution.

Example

$x = 10 + 5 * 2$

Evaluation:

$5 * 2 = 10$

$10 + 10 = 20$

Final result

$x = 20$

Using Parentheses

Parentheses can change evaluation order.

Example

$x = (10 + 5) * 2$

Evaluation

$10 + 5 = 15$

$15 * 2 = 30$

Result

$x = 30$

Parentheses improve **clarity and readability**.

Order of Evaluation of Subexpressions

C does not guarantee the order in which subexpressions are evaluated.

Example

$x = f1() + f2()$

The compiler may execute:

$f1()$ first

or

$f2()$ first

Therefore, programmers should **avoid writing expressions that depend on evaluation order**.

Type Conversion in Expressions

When different data types appear in an expression, C automatically converts them into a common type.

This process is called **type conversion**.

Example

`int a = 10;`

`float b = 5.5;`

`a + b`

Conversion:

`int` $\rightarrow$ `float`

Result type:

`float`

Type Promotion

Before evaluating expressions, C promotes smaller data types to larger ones.

Example

char $\rightarrow$ int
short $\rightarrow$ int

This process is called **integral promotion**.

Example

```
char ch = 'A';  
int x;  
x = ch + 5;
```

Conversion

'A' $\rightarrow$ ASCII value 65
 $65 + 5 = 70$

Usual Arithmetic Conversion

C follows certain rules for type conversion.

Conversion Rules

1. If one operand is **long double** $\rightarrow$ other converted to long double
2. Else if one operand is **double** $\rightarrow$ other converted to double
3. Else if one operand is **float** $\rightarrow$ other converted to float
4. Else integers are promoted

Example

```
int a = 5  
double b = 3.2  
a + b
```

Conversion

int $\rightarrow$ double

Result type

double

Example of Type Conversion

```
#include<stdio.h>  
int main()
```

```
{
    int a = 5;
    float b = 2.5;
    float result;
    result = a + b;
    printf("%f", result);

    return 0;
}
```

Output

7.500000

Explanation

a → converted to float
 $5.0 + 2.5 = 7.5$

Type Casting

Sometimes programmers need to **explicitly convert data types**.

This is done using **type casting**.

Type casting is the **explicit conversion of one data type into another** using a cast operator.

Syntax

(type) expression

Example

(float) x

Example of Type Casting

```
#include<stdio.h>
int main()
{
    int x = 5;

    printf("%f", (float)x/2);

    return 0;
}
```

Output

2.500000

Without Type Casting

$5 / 2 = 2$

(integer division)

With Type Casting

(float) $5 / 2 = 2.5$

Importance of Type Casting

Type casting is useful for:

1. Preventing loss of precision
2. Performing floating-point division
3. Controlling type conversions
4. Improving program accuracy

Example Demonstrating Casting

```
#include<stdio.h>
```

```
int main()
{
    int total = 17;
    int count = 4;

    float average;

    average = (float) total / count;

    printf("Average = %f", average);

    return 0;
}
```

Output

Average = 4.250000

Without casting

$17 / 4 = 4$

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and
student-focused resources for every semester.

Connect With Us:

 www.azdocuments.in [[Click Here](#)]

 [@azdocuments](#) [[Click Here](#)]

 Join Our Student Community: [[Click Here](#)]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.